



Universidad Nacional Pedro Ruiz Gallo
Facultad de Ingeniería Civil, de Sistemas y de Arquitectura
Escuela Profesional de Ingeniería de Sistemas

LogiLearn

Lógica Simbólica Global

Informe Técnico y Documentación de Sprints
Arquitectura, motores lógicos y evolución en 4 semanas · 4 equipos

Curso:	MATG1001 — 2026 I, II Ciclo (3 créditos)
Docente:	Dr. Mardo Victor Gonzales Herrera
Líder:	Enrique Díaz Cayaca
Equipos:	Sinergia · Hijos de Linus · Modus Innova · Linus
Stack:	Vue 3 + Vite 6 + TypeScript + Tailwind v4 + Vitest (189 pruebas)
Repositorio:	https://github.com/EnriDiazCayaca/logica_simbolica_global

Lambayeque, Perú — Septiembre 2026

Índice general

1. Resumen y alcance	1
2. Marco teórico	2
2.1. Lógica proposicional	2
2.2. Inferencia	2
2.3. Cuantificadores	2
2.4. Teoría de conjuntos	2
3. Metodología y organización	3
3.1. Cuatro equipos	3
3.2. Flujo Git y sprints	3
4. Arquitectura del software	4
5. Motor de tablas de verdad	6
5.1. Tipos y árbol de sintaxis	6
5.2. Normalización y tokenizador	6
5.3. Parser descendente recursivo	7
5.4. Evaluación y generación de filas	7
6. Motor de inferencias	9
6.1. Parser del solver	9
6.2. Evaluación y contraejemplo	9
6.3. Diez reglas por reconocimiento de patrones	9
6.4. Diagnóstico de falacias	9
6.5. Encadenamiento hacia adelante	9
7. Motor de cuantificadores	11
7.1. Tokenizador y parser de predicados	11
7.2. Dominio $D \subset \mathbb{Z}$	11
7.3. Evaluación y leyes de reescritura	12
8. Motor de conjuntos	14
8.1. Operaciones puras	14
9. Diez reglas de inferencia en detalle	16
9.1. Modus Ponens y Modus Tollens	16
9.2. Silogismos	16
9.3. Adición, simplificación y conjunción	16
9.4. Doble negación y dilemas	16
10. Diagnóstico de falacias con ejemplos	17

11.Catálogo de complejidad por función	18
12.Comparación de los dos parsers	19
13.Diario de sprints por equipo	20
13.1. Sprint 1 — Propuesta	20
13.2. Sprint 2 — Motores	20
13.3. Sprint 3 — Interfaz	21
13.4. Sprint 4 — Refinamiento y calidad	21
14.Catálogo de pruebas por archivo	23
15.Leyes formales con demostración esquemática	24
16.Módulos de validación, trazabilidad y transcripción	25
17.Páginas Vue y sistema de contenido	26
18.Integración Vue y contenido externalizado	27
19.Pruebas, calidad y despliegue	29
20.Evolución por sprints	30
21.Conclusiones	31
22.Anexo: bitácoras de sprint (transcripción fiel formateada)	32
22.1. Propuesta Sinergia	32
22.2. LogiLearn	32
22.3. Propuesta Hijos de Linus	33
22.4. Asistente e Intérprete de Inferencias Lógicas	33
22.5. Propuesta Linus	34
22.6. Asistente e Intérprete de Teoría de Conjuntos y Diagramas de Venn	34
22.7. Propuesta Modus Innova	36
22.8. Validador y Tutor Interactivo de Cuantificadores y Silogismos Lógicos (QuantifiWeb)	36
22.9. Avance Sprint 2 Hijos de Linus	38
22.10 Avances del Sprint 2 - Equipo: Hijos de Linus	38
22.10.1 Integrante: Arom	38
22.10.2 Integrantes: Morocho y Alex	39
22.10.3 Integrante: Mio	40
22.10.4 Integrante: Renato	41
22.11 Casos de prueba Hijos de Linus	42
22.12 Matriz de Casos Borde y Pruebas (Módulo de Validación)	42
22.12.1.1. Clasificación y Matriz de Casos	42
22.12.2.2. Estrategia de Implementación	43
22.13 Avance Sprint 2 Linus	43
22.14 Avance del Motor Lógico - Equipo Linus (Conjuntos)	43
22.14.1. Qué hicimos	43
22.14.2. Archivos Creados	43
22.14.3. Qué falta	43
22.14.4. Cómo usar el motor (Ejemplo)	44
22.15 Avance Sprint 3 Hijos de Linus	44
22.16 Registro de Avances — Los Hijos de Linus	44
22.16.1 SPRINT 3.5 (Completado)	44

22.16.2SPRINT 3 (Completado)	45
22.17Sprint 3.5 Hijos de Linux	45
22.18Sprint 3.5: Perfeccionamiento y Refinamiento (Inferencias y UI)	46
22.18.1Responsables y Estado de Tareas	46
22.18.2Archivo de Historial	46
22.19Avance Sprint 3 Linux	46
22.20Avance de Interfaz Visual - Equipo Linux (Conjuntos)	46
22.20.1.Qué hicimos	46
22.20.2.Archivos Modificados	47
22.20.3.Qué falta	47
22.20.4.Cómo probar el módulo	47
22.21Errores y soluciones Hijos de Linux	47
22.22Registro de Errores, Diagnósticos y Soluciones — Los Hijos de Linux	47
22.22.1.Resumen Ejecutivo	48
22.22.2.1. Contradicción Semántica: Falso Positivo de 'Inferencia Inválida' en Argumentos Válidos con Prueba Indirecta	48
22.22.3.2. Ausencia de Contraejemplos Formales en Argumentos Inválidos y Falacias	48
22.22.4.3. Soporte y Semántica de la Disyunción Fuerte / Exclusiva (/ XOR)	49
22.22.5.4. Desbordamiento y Botones Huérfanos en Teclado Móvil	49
22.22.6.5. Apertura Involuntaria del Teclado Nativo Móvil (Soft Keyboard)	49
22.22.7.6. Espaciado Automático Inconsistente en Paréntesis y Variables	50
22.22.8.7. Desplazamiento Visual por Banner de Linter en Tiempo Real	50
22.22.9.8. Escala y Redimensionamiento Dispar de Glifos Unicode en Android	50
22.22.10. Inconsistencia de Sintaxis LaTeX en la Exportación a Markdown	51
22.22.11.0. Duplicación de Rutas Deductivas en Silogismo Hipotético (SH)	51
22.22.12.1. Explicaciones Didácticas con Variables Abstractas Genéricas	51
22.22.13Matriz de Estado Final	52
22.23Reporte de errores de inferencias	52
22.24Reporte de Errores — Módulo de Inferencias	52
22.24.1.Resumen ejecutivo	52
22.24.2.Error 1 — Paso redundante en la demostración	52
22.24.3.Error 2 — Pasos redundantes y duplicados (caso con 4 premisas)	53
22.24.4.Sugerencia técnica para resolver el problema	53
22.24.5.Casos de prueba sugeridos	55
22.25Uso del modulo de inferencias	56
22.26Uso del Módulo: Demostrador de Inferencias Lógicas	56
22.26.1.Cómo funciona	56
22.26.2.Ejemplos	57
22.26.3.Operadores Soportados	58
22.26.4.Limitaciones y Alcance	58
22.26.5.Verificación de Calidad y Tests	59
22.27Uso del modulo de conjuntos	59
22.28Uso del Módulo: Teoría de Conjuntos y Diagramas de Venn	59
22.28.1.Cómo funciona	59
22.28.2.Ejemplos	59
22.28.3.Cómo probar en la app	60
22.28.4.Cómo correr los tests	60
22.28.5.Limitaciones	60
22.29Estado de equipos	60
22.30Estado de los Equipos — Contexto para Agentes de Guía	60
22.30.1.Distribución del Sílabo (4 Equipos)	60

22.30.2.Estado por Equipo	61
22.30.3.Observaciones Importantes	61
22.30.4.Entregables por Sprint (Referencia)	62
22.30.5.Stack Tecnológico	62
22.30.6.Comandos Útiles	62
22.30.7.Log de Cambios	62
23.Catálogo de pruebas por archivo	63
24.Leyes formales del sistema	64
25.Galería técnica complementaria	65
25.1. Motor de tablas	65
25.2. Motores de conjuntos y cuantificadores	73
26.Validación, trazabilidad y páginas	76
27.Registro de decisiones de diseño	77
28.Despliegue y mantenimiento	78
28.1. Procedimiento paso a paso	78
29.Glosario técnico	79
30.Casos de prueba nominales por archivo	80
31.Etapas del reescritor con ejemplo	83
32.Demostraciones formales por tabla	84
32.1. Conectivos primitivos	84
32.2. De Morgan proposicional y cuantificacional	84
32.3. Leyes de conjuntos por extensión	84
33.Retrospectiva por equipo	85
34.Sistema de contenido por módulo	86
35.Categorías del validador en resumen	87
36.Guía de lectura y trabajo futuro	88
36.1. Cómo leer este informe	88
36.2. Trabajo futuro	88
37.Índice de correspondencia de figuras	89
38.Matriz de trazabilidad requisito-evidencia	91

Índice de figuras

3.1. Cronograma del equipo Sinergia (material original)	3
4.1. Estructura de archivos del proyecto (material original Sinergia)	5
5.1. Gramática del motor proposicional (material original Sinergia)	6
5.2. Tokenizador y normalización (material original Sinergia)	6
5.3. Parser descendente recursivo (material original Sinergia)	7
5.4. Generación de filas por bits (material original Sinergia)	8
6.1. Boceto del árbol sintáctico (archivo original Hijos de Linus)	10
7.1. Validación del dominio entero (material original Modus Innova)	12
7.2. Regla de disyunción fuerte (material original Modus Innova)	13
8.1. Diagrama de Venn interactivo (material original Linus)	15
13.1. Propuestas Fernando y Mike (Linus, archivos originales)	20
13.2. Parser del equipo Sinergia, detalle (material original)	21
13.3. Propuestas Nio y Sergio (Linus, archivos originales)	21
13.4. Entorno de práctica del equipo Hijos de Linus (archivo original)	22
15.1. Demostración paso a paso del motor de leyes (Modus Innova, material original) .	24
17.1. Propiedades computadas de la vista (material original Sinergia)	26
18.1. Estado reactivo de la vista de tablas (material original Sinergia)	28
19.1. Cronograma de sprints (material original Sinergia)	29
20.1. Bocetos del equipo Linus (archivos originales)	30
20.2. Interfaz del equipo Hijos de Linus (archivo original)	30
25.1. Portada del informe original Sinergia	65
25.2. Tipos del evaluador (material original Sinergia)	65
25.3. Tokenizador autómatas (material original Sinergia)	66
25.4. Parser, vista larga (material original Sinergia)	67
25.5. Parser recursivo en detalle (material original Sinergia)	68
25.6. Evaluación semántica, continuación (material original Sinergia)	68
25.7. Evaluación y ramas (material original Sinergia)	69
25.8. Envoltura del árbol a Unicode (material original Sinergia)	69
25.9. Estado reactivo de la vista (material original Sinergia)	70
25.10 Estado reactivo, continuación (material original Sinergia)	70
25.11 Propiedades computadas (material original Sinergia)	71
25.12 Acceso documentado (material original Sinergia)	71

25.13Pruebas del evaluador (material original Sinergia)	71
25.14Árbol del repositorio (material original Sinergia)	72
25.15Boceto inicial de conjuntos (material original Linus)	73
25.16Flujo Git del equipo Linus (material original)	73
25.17Trabajo colaborativo Linus (material original)	74
25.18Boceto QuantifiWeb (material original Modus Innova)	74
25.19Motor de cuantificadores (material original Modus Innova)	74
25.20Integración Vue (material original Modus Innova)	75
25.21Leyes, segunda parte (material original Modus Innova)	75
26.1. Logo institucional LogiLearn	76

Índice de tablas

2.1. Conectivos y condición de verdad	2
3.1. Equipos y responsabilidades	3
4.1. Arquitectura en tres capas	4
6.1. Reglas implementadas	9
8.1. Potencia $P(\{1, 2\})$ por máscara binaria	14
9.1. Reglas y costo de reconocimiento	16
11.1. Complejidad de las funciones principales	18
12.1. Gramáticas comparadas	19
14.1. Archivos de prueba y cobertura	23
18.1. Trazabilidad pantalla-contenido (selección)	27
23.1. Archivos de prueba y cobertura	63
27.1. Ocho decisiones con alternativa descartada	77
30.1. Casos nominales (selección de 6 por archivo)	80
30.1. Casos nominales (continuación)	81
30.1. Casos nominales (continuación)	82
31.1. Ocho etapas con transformación de ejemplo	83
32.1. Tablas de los cinco conectivos	84
34.1. Claves de contenido por módulo	86
37.1. Figuras y fuentes	89
37.1. Figuras y fuentes (continuación)	90
38.1. Requisito y evidencia	91

Capítulo 1

Resumen y alcance

LogiLearn implementa el cien por ciento del sílabo MATG1001 con motores reales: tablas de verdad con clasificación, inferencias con trazabilidad y contraejemplo de 2^N combinaciones, cuantificadores $\forall$ y $\exists$ sobre dominios finitos $D \subset \mathbb{Z}$ con De Morgan y disyunción fuerte Δ , y conjuntos con diagrama de Venn interactivo. Este informe documenta qué se construyó, cómo se construyó (Vue 3 con componentes, motores aislados en `src/lib` y contenido externalizado por módulo con interruptor de modo literal) y cuándo (cuatro sprints). Las fuentes primarias son los informes de cada equipo, unificados aquí con voz única y terminología común: $D \subset \mathbb{Z}$ siempre denota dominio finito de enteros y los conectivos se escriben $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$.

Capítulo 2

Marco teórico

Lógica proposicional

Una proposición toma valor V o F . Los conectivos son negación $\neg$, conjunción $\wedge$, disyunción $\vee$, implicación $\rightarrow$ y bicondicional $\leftrightarrow$. Una fórmula es *tautología* si es V en toda asignación, *contradicción* si es F en toda asignación y *contingencia* en otro caso. Para n variables hay 2^n filas.

Tabla 2.1: Conectivos y condición de verdad

Conectivo	Símbolo	Es F cuando
Negación	$\neg p$	p es V
Conjunción	$p \wedge q$	alguna es F
Disyunción	$p \vee q$	ambas son F
Condicional	$p \rightarrow q$	p es V y q es F
Bicondicional	$p \leftrightarrow q$	difieren

Inferencia

Una inferencia es válida si, siendo verdaderas las premisas, la conclusión es necesariamente verdadera: Modus Ponens $p, p \rightarrow q \vdash q$, Modus Tollens $\neg q, p \rightarrow q \vdash \neg p$ y Silogismo Hipotético $p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$. Las falacias de afirmar el consecuente y negar el antecedente son inválidas.

Cuantificadores

$\forall x P(x)$ es verdadero si $P(x)$ lo es para todo x en D ; $\exists x P(x)$ si hay al menos un testigo. De Morgan:

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x), \quad \neg(\exists x P(x)) \equiv \forall x \neg P(x). \quad (2.1)$$

Teoría de conjuntos

Unión $A \cup B$, intersección $A \cap B$, diferencia $A \setminus B$, complemento A^c y diferencia simétrica $A \Delta B \equiv (A \cup B) \setminus (A \cap B)$. De Morgan conjuntista: $(A \cup B)^c = A^c \cap B^c$. La potencia cumple $|P(A)| = 2^{|A|}$.

Capítulo 3

Metodología y organización

Cuatro equipos

Tabla 3.1: Equipos y responsabilidades

Equipo	Sublíder	Tema	Carpeta
Sinergia	Alexa	Tablas	<code>src/pages/tablas</code>
Hijos de Linus	Arom	Inferencias	<code>src/pages/inferencias</code>
Modus Innova	Cristian	Cuantificadores	<code>src/pages/cuantificadores</code>
Linus	Jordy	Conjuntos	<code>src/pages/conjuntos</code>

Flujo Git y sprints

Ramas por funcionalidad con Pull Requests revisados por el líder; integración final en `main` sin resaltado experimental. Cuatro sprints: propuesta con wireframes, motores con pruebas, interfaz con componentes compartidos, y calidad con 189 pruebas, verificación de tipos y despliegue en Pages con vistas previas por Pull Request. La evidencia por sprint vive en `TAREAS/01_Propuesta` a `TAREAS/04_Deploy` con mapa y cronograma.

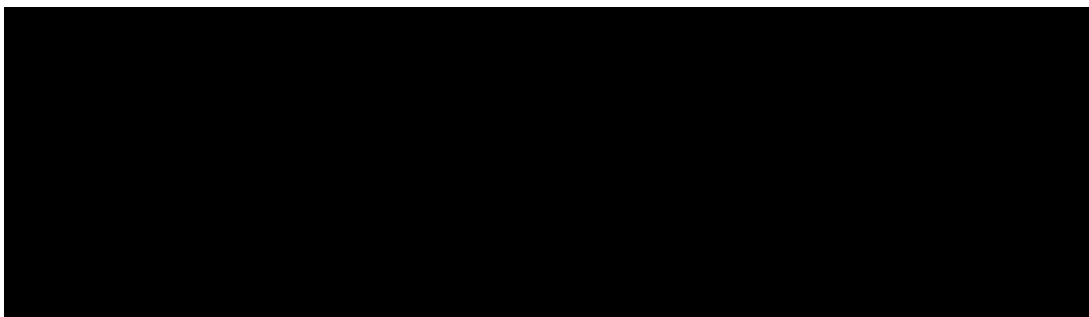


Figura 3.1: Cronograma del equipo Sinergia (material original)

Capítulo 4

Arquitectura del software

La lógica vive en `src/lib` (motores puros y testeables) separada de `src/pages` (componentes) y de los componentes compartidos de interfaz. El enrutador define las rutas y el almacén de progreso persiste en el navegador. El contenido visible se externaliza por módulo con interruptor de modo literal que elige símbolos o palabras sin tocar código.

Tabla 4.1: Arquitectura en tres capas

Capa	Tecnología y responsabilidad
Presentación	Vue 3 con Composition API; reactividad con referencias y computadas; diagrama Venn en SVG
Lógica	TypeScript puro en <code>src/lib</code> : unión, intersección, diferencia, complemento y potencia; sin dependencias externas, totalmente testeable
Estado	Variables reactivas de Vue; estado local del componente sin almacén global; sincronización automática entre interfaz y lógica

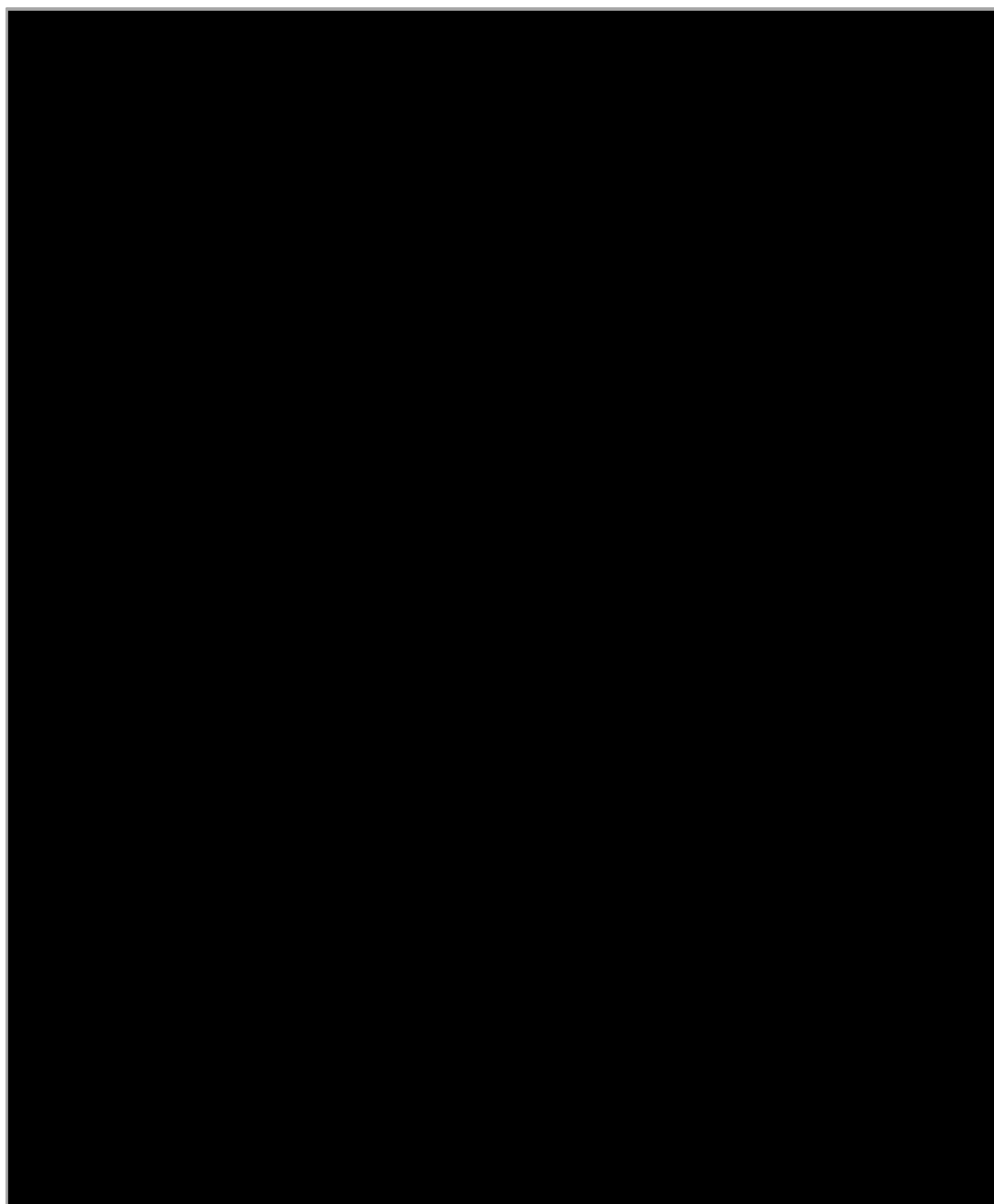


Figura 4.1: Estructura de archivos del proyecto (material original Sinergia)

Capítulo 5

Motor de tablas de verdad

Tipos y árbol de sintaxis

El motor representa cada fórmula como un árbol: los nodos internos son operadores y las hojas son variables. La precedencia implementada es $\neg > \wedge > \vee > \rightarrow > \leftrightarrow$, de modo que $P \wedge Q \rightarrow R$ se lee $(P \wedge Q) \rightarrow R$.

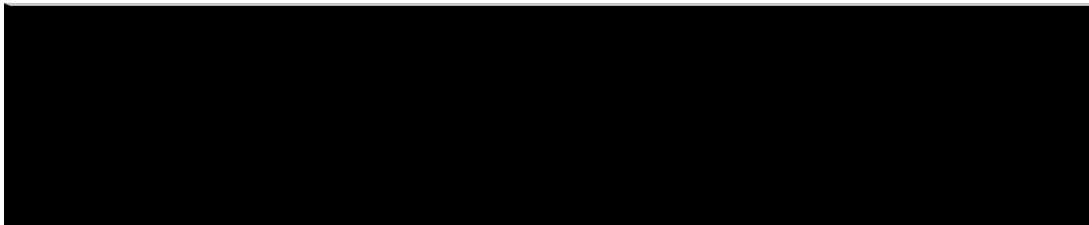


Figura 5.1: Gramática del motor proposicional (material original Sinergia)

Normalización y tokenizador

La entrada acepta Unicode, ASCII y palabras (NOT, AND, $\rightarrow$). El normalizador unifica variantes antes del análisis léxico de complejidad $O(n)$.

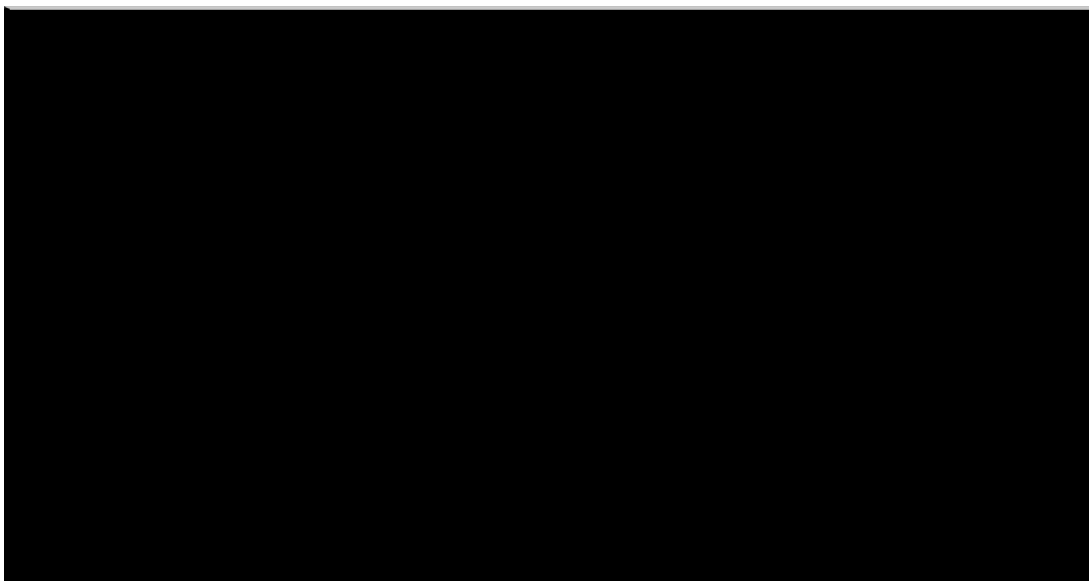


Figura 5.2: Tokenizador y normalización (material original Sinergia)

Parser descendente recursivo

Cada nivel de precedencia es una función que llama al siguiente; la asociatividad a izquierda se logra con bucles. El esquema esencial es:

Listing 5.1: Esquema del parser por niveles

```
1 function parseIff() { let n = parseImplies(); /* bucle */ return n; }
2 function parseImplies() { let n = parseOr(); /* bucle */ return n; }
3 function parseOr() { let n = parseAnd(); /* bucle */ return n; }
4 function parseAnd() { let n = parseNot(); /* bucle */ return n; }
5 function parseNot() { /* recursivo para doble negacion */ }
```

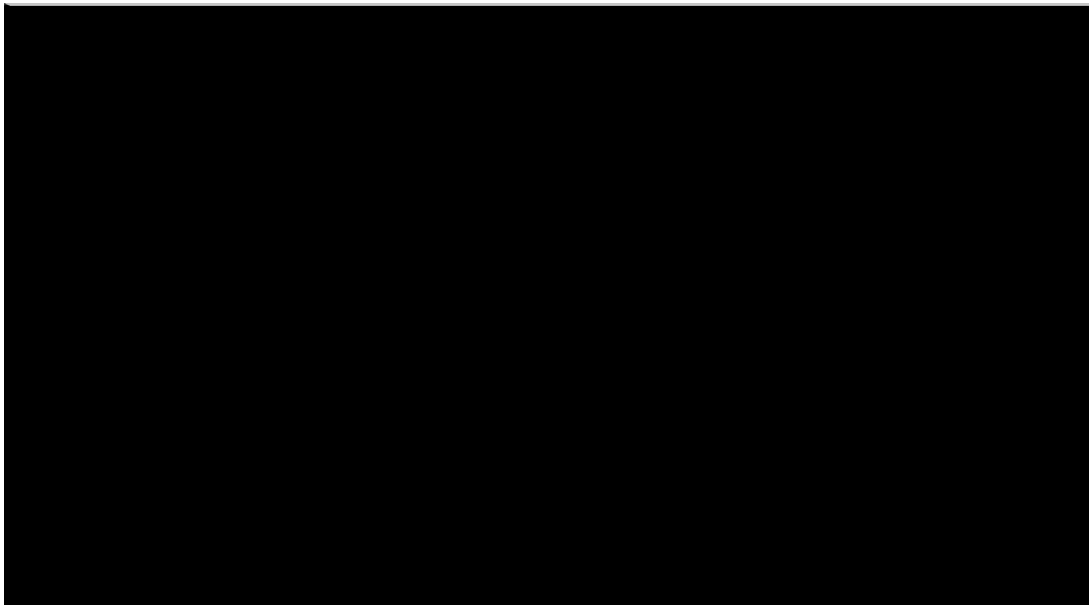


Figura 5.3: Parser descendente recursivo (material original Sinergia)

Evaluación y generación de filas

La evaluación recorre el árbol en $O(\text{nodos})$ con semántica material ($\rightarrow$ es $\neg p \vee q$). La generación produce 2^v filas por aritmética de bits, de costo $O(2^v \cdot n)$:

Listing 5.2: Esquema de generación de filas

```
1 const total = 1 << vars.length; // 2^v combinaciones
2 for (let i = 0; i < total; i++) {
3   // bit (v-1-j) de i define V/F de vars[j]
4 }
```

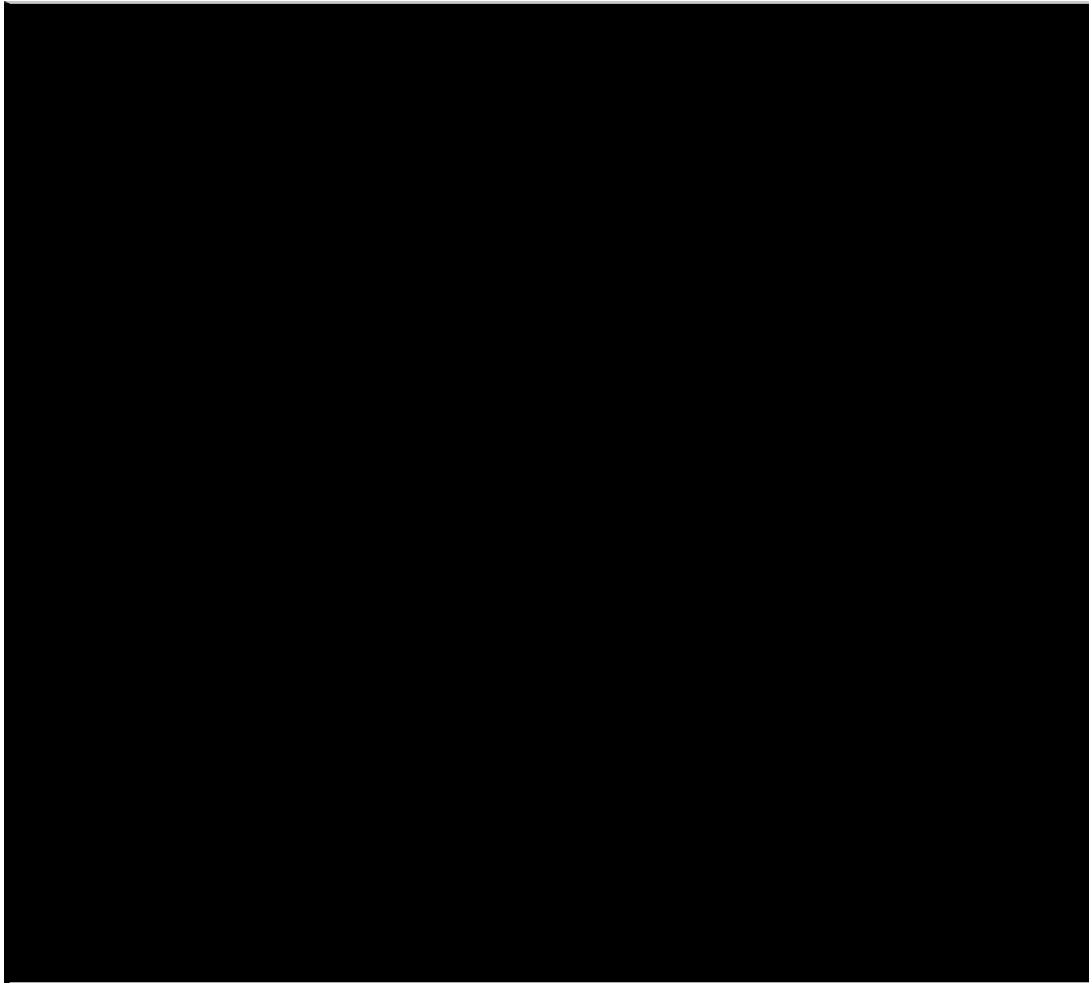


Figura 5.4: Generación de filas por bits (material original Sinergia)

La clasificación cuenta verdaderas y falsas: todo V es tautología, todo F es contradicción y lo mixto es contingencia.

Capítulo 6

Motor de inferencias

Parser del solver

El analizador del solver trabaja con palabras formales en español (*Y*, *O*, *NO*, *ENTONCES*) y cuatro niveles de precedencia. La negación es recursiva a derecha para $\neg\neg p$.

Evaluación y contraejemplo

Con v variables se exploran 2^v asignaciones: hay contraejemplo cuando las premisas son V y la conclusión es F ; hay inconsistencia cuando ninguna asignación hace V a todas las premisas. Complejidad $O(2^v \cdot p \cdot n)$.

Diez reglas por reconocimiento de patrones

Cada regla es $O(n)$ sobre el árbol: Modus Ponens, Modus Tollens, bicondicionales, silogismos disyuntivo e hipotético, simplificación, doble negación y dilema constructivo (ternaria, $O(n^3)$ en su búsqueda).

Tabla 6.1: Reglas implementadas

Regla	Esquema
Modus Ponens	$p, p \rightarrow q \vdash q$
Modus Tollens	$\neg q, p \rightarrow q \vdash \neg p$
Silogismo hipotético	$p \rightarrow q, q \rightarrow r \vdash p \rightarrow r$
Silogismo disyuntivo	$p \vee q, \neg p \vdash q$
Adición / Simplificación	$p \vdash p \vee q / p \wedge q \vdash p$

Diagnóstico de falacias

El diagnóstico en cascada prioriza falacias formales sobre el resultado semántico: variable ausente, afirmación del consecuente, negación del antecedente, dilema inverso, contraejemplo general, inconsistencia e incompletitud. Es el diferencial pedagógico del módulo.

Encadenamiento hacia adelante

El demostrador itera hasta 12 veces con tres fases por iteración (unarias, binarias $O(n^2)$, ternarias $O(n^3)$), deduplicando fórmulas conocidas y podando pasos redundantes por retroceso

desde la conclusión:

Listing 6.1: Esquema del encadenamiento

```

1 for (let it = 0; it < 12; it++) {
2   aplicarReglasUnarias();    //  $O(n)$ 
3   aplicarReglasBinarias();   //  $O(n^2)$ 
4   aplicarReglasTernarias();  //  $O(n^3)$ 
5   if (conclusionAlcanzada()) break;
6 }
7 podarPasosRedundantes();

```

Mis ejercicios

+ Nuevo ejercicio

GUARDADOS

- De Morgan 1 (06/08/2026 · 10:24) ✓
- Silogismo hipotético (05/08/2026 · 18:40) ✓
- Ejercicio 1 (04/08/2026 · 21:15) ✓
- Ejercicio 2 (03/08/2026 · 16:22) ✓
- Práctica 5 (02/08/2026 · 11:05) ✓
- Ejercicio parcial (01/08/2026 · 09:33) ⚪

Eliminar seleccionado

Nuevo ejercicio
Ingresa tus premisas en lenguaje natural y obtén su representación simbólica.

Ingresa premisa

Ejemplo: no(p) o q, p implica r, todos los A son B...

+ Agregar premisa

Al agregar, la premisa se convertirá a notación simbólica y aparecerá en la lista.

Premisas ingresadas

Limpiar todo

- 1. $\neg p \vee q$
- 2. $p \rightarrow r$
- 3. q

Resolución

1. $\neg p \vee q$ (no(p) o q)
 2. $p \rightarrow r$ (p implica r)
 3. q (q)

Conclusión:
 $\neg p$

Guardar ejercicio

Figura 6.1: Boceto del árbol sintáctico (archivo original Hijos de Linus)

Capítulo 7

Motor de cuantificadores

Tokenizador y parser de predicados

Seis niveles de precedencia con encadenamiento de comparaciones ($a < b < c$ equivale a $(a < b) \wedge (b < c)$) y potencia con `Math.pow`. Complejidad $O(n)$.

Dominio $D \subset \mathbb{Z}$

El dominio acepta listas (1, 2, 3) y rangos (0 < x < 9 equivale a $D = \{x \in \mathbb{Z} : 0 < x < 9\}$) con cota de 2000 elementos y deduplicación. La variante estricta exige solo la variable x y solo enteros.

```
// MÓDULO 1: EVALUADOR DE CUANTIFICADORES (quantifierEngine.ts)
export function parseDomain(domainRaw: string): number[] {
  const partes = domainRaw.split(',').map((s) => s.trim()).filter(Boolean);
  if (partes.length === 0) throw new ExpresionInvalidaError('El dominio D no puede estar vacío.');
```

```
  const valores: number[] = [];
  for (const parte of partes) {
    const match = parte.match(/^(?!\s*(-?\d+(?:\.\d+)?)\s*(<=|<)\s*x\s*(<=|<)\s*(-?\d+(?:\.\d+)?)\s*\)\s*/i);
    if (match) {
      const lower = Number(match[1]);
      const upper = Number(match[4]);
      if (!Number.isInteger(lower) || !Number.isInteger(upper)) {
        throw new ExpresionInvalidaError('El dominio D opera únicamente sobre números enteros (Z).');
```

```
      }
      const inicio = match[2] === '<=' ? Math.ceil(lower) : Math.floor(lower) + 1;
      const fin = match[3] === '<=' ? Math.floor(upper) : Math.ceil(upper) - 1;
      for (let i = inicio; i <= fin; i++) valores.push(i);
      continue;
    }
    if (parte.includes('.')) throw new ExpresionInvalidaError('El elemento "${parte}" es decimal; D ⊂ Z solo acepta enteros.');
```

```
    const num = Number(parte);
    if (!Number.isInteger(num)) throw new ExpresionInvalidaError(`"${parte}" no es un número entero válido.`);
    valores.push(num);
  }
  return Array.from(new Set(valores));
}
```

Figura 7.1: Validación del dominio entero (material original Modus Innova)

Evaluación y leyes de reescritura

Sobre $|D| = m$ elementos el costo es $O(m \cdot n)$ con trazabilidad por elemento, contraejemplo y testigo. El reescritor aplica ocho reglas en orden fijo (doble negación, Δ , bicondicional, implicación, De Morgan, distribución) hasta 15 iteraciones:

```
// MÓDULO 2: MOTOR DE REESCRITURA Y LEYES (lawsEngine.ts)
export function tryApplyRule(ruleId: RuleId, formula: string): ProofStep | null {
  switch (ruleId) {
    case 'disyuncion_fuerte': { // REGLA:  $A \Delta B \equiv [(A \vee B) \wedge \sim(A \wedge B)]$ 
      for (let i = 0; i < formula.length; i++) {
        if (formula[i] === 'Δ' || formula[i] === '⊕') {
          const leftObj = getOperandLeft(formula, i);
          const rightObj = getOperandRight(formula, i + 1);
          if (leftObj && rightObj) {
            const A = leftObj.text; const B = rightObj.text;
            const replacement = `[(${A} ∨ ${B}) ∧ ∼(${A} ∧ ${B})]`;
            return {
              formula: formula.substring(0, leftObj.start) + replacement +
formula.substring(rightObj.end),
              rule: 'Definición de Disyunción Fuerte (Exclusiva)',
              changedChunk: replacement
            };
          }
        }
      }
      break;
    }
  }
  return null;
}
```

Figura 7.2: Regla de disyunción fuerte (material original Modus Innova)

Capítulo 8

Motor de conjuntos

Operaciones puras

Funciones genéricas sobre conjuntos nativos: unión $O(n + m)$ por propagación con deduplicación, resto $O(n)$ por filtrado con pertenencia $O(1)$:

Listing 8.1: Núcleo de operaciones

```
1 function union(a, b) { return new Set([...a, ...b]); }  
2 function interseccion(a, b) { return new Set([...a].filter(x => b.has(x))); }
```

La potencia usa máscara de bits sobre 2^n iteraciones ($O(2^n \cdot n)$), lo que justifica la advertencia con $|A| > 10$ en la interfaz.

Tabla 8.1: Potencia $P(\{1, 2\})$ por máscara binaria

Máscara	Subconjunto
00	$\emptyset$
01	$\{1\}$
10	$\{2\}$
11	$\{1, 2\}$

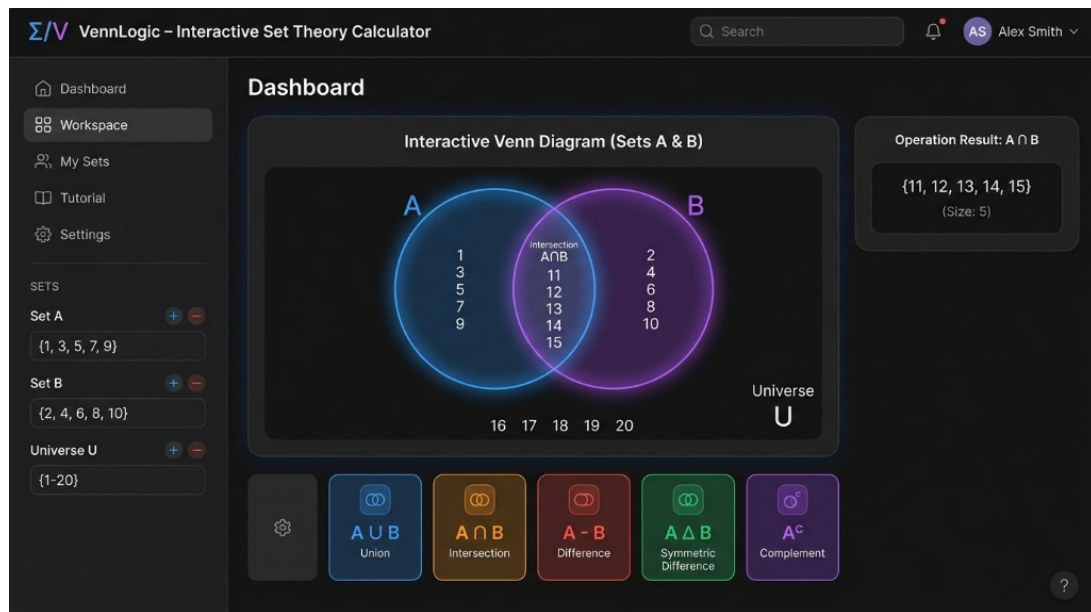


Figura 8.1: Diagrama de Venn interactivo (material original Linus)

Capítulo 9

Diez reglas de inferencia en detalle

Cada regla indica esquema formal, patrón de reconocimiento sobre el árbol y ejemplo. Todas son $O(n)$ sobre el tamaño del árbol salvo el dilema ($O(n^3)$ en búsqueda).

Modus Ponens y Modus Tollens

Modus Ponens: de p y $p \rightarrow q$ se deduce q . El patrón busca una implicación cuyo antecedente coincida estructuralmente con una premisa. Modus Tollens: de $\neg q$ y $p \rightarrow q$ se deduce $\neg p$. Ejemplo: con $P \rightarrow Q$ y $\neg Q$ se deriva $\neg P$.

Silogismos

El hipotético encadena $p \rightarrow q$ y $q \rightarrow r$ en $p \rightarrow r$ por transitividad. El disyuntivo resuelve $p \vee q$ con $\neg p$ en q . La versión exclusiva opera sobre Δ con paridad exacta.

Adición, simplificación y conjunción

La adición construye $p \vee q$ desde p ; la simplificación extrae p desde $p \wedge q$; la conjunción une dos premisas independientes en $p \wedge q$.

Doble negación y dilemas

La doble negación cancela $\neg\neg p$ en p . El dilema constructivo combina $p \rightarrow q$, $r \rightarrow s$ y $p \vee r$ en $q \vee s$. La eliminación del bicondicional descompone $p \leftrightarrow q$ en ambos condicionales.

Tabla 9.1: Reglas y costo de reconocimiento

Regla	Patrón	Costo
Modus Ponens	implicación + antecedente	$O(n)$
Modus Tollens	implicación + negación del consecuente	$O(n)$
Silogismo hipotético	dos implicaciones encadenadas	$O(n)$
Silogismo disyuntivo	disyunción + negación	$O(n)$
Adición / Simplificación	construcción / proyección	$O(n)$
Dilema constructivo	dos implicaciones + disyunción	$O(n^3)$

Capítulo 10

Diagnóstico de falacias con ejemplos

El diagnóstico en cascada prioriza: variable ausente, afirmación del consecuente ($p \rightarrow q, q \vdash p$ con contraejemplo $p = F, q = V$), negación del antecedente ($p \rightarrow q, \neg p \vdash \neg q$), dilema inverso, contraejemplo general, inconsistencia e incompletitud. Cada diagnóstico cita la asignación que refuta y la regla vulnerada, con valor pedagógico directo.

Capítulo 11

Catálogo de complejidad por función

Tabla 11.1: Complejidad de las funciones principales

Función	Módulo	Complejidad
<code>union</code>	conjuntos	$O(n + m)$
<code>interseccion</code>	conjuntos	$O(n)$
<code>diferencia</code>	conjuntos	$O(n)$
<code>potencia</code>	conjuntos	$O(2^n \cdot n)$
<code>normalizar</code>	tablas	$O(n)$
<code>tokenizar</code>	tablas	$O(n)$
<code>parsear*</code> (5 niveles)	tablas	$O(n)$
<code>evaluar</code>	tablas	$O(\text{nodos})$
<code>generarFilas</code>	tablas	$O(2^v \cdot n)$
<code>encontrarContraejemplo</code>	inferencias	$O(2^v \cdot p \cdot n)$
<code>demostrarConclusion</code>	inferencias	$O(12 \cdot (n + n^2 + n^3))$
<code>tokenizeExpression</code>	cuantificadores	$O(n)$
<code>parseDomain</code>	cuantificadores	$O(n)$
<code>evaluarCuantificador</code>	cuantificadores	$O(m \cdot n)$
<code>solveFormula</code>	leyes	$O(15 \cdot r \cdot n)$

Los límites explícitos (2000 elementos de dominio, 12 iteraciones, advertencia con $|A| > 10$) evitan la explosión combinatoria.

Capítulo 12

Comparación de los dos parsers

El parser de tablas usa escáner por caracteres con símbolos Unicode y cinco niveles; el del solver usa escáner por palabras con español formal y cuatro niveles. Ambos son descendentes recursivos $O(n)$ con recursión derecha para la negación.

Tabla 12.1: Gramáticas comparadas

Tablas	Solver
Iff $\leftrightarrow$	Implicación y bicondicional
Implies $\rightarrow$	Disyunciones
Or $\vee$	Conjunción <i>Y</i>
And $\wedge$	Negación <i>NO</i>
Not $\neg$	Primario
Primario	—

Capítulo 13

Diario de sprints por equipo

Sprint 1 — Propuesta

Cada equipo entregó wireframes y decisión de pila: Sinergia propuso el motor proposicional con precedencia clásica; Hijos de Linus el demostrador con trazabilidad; Modus Innova el evaluador sobre $D \subset \mathbb{Z}$ con Δ ; Linus la calculadora con Venn. La plantilla base Vue 3 + Vite se creó con la estructura que se mantuvo todo el desarrollo.

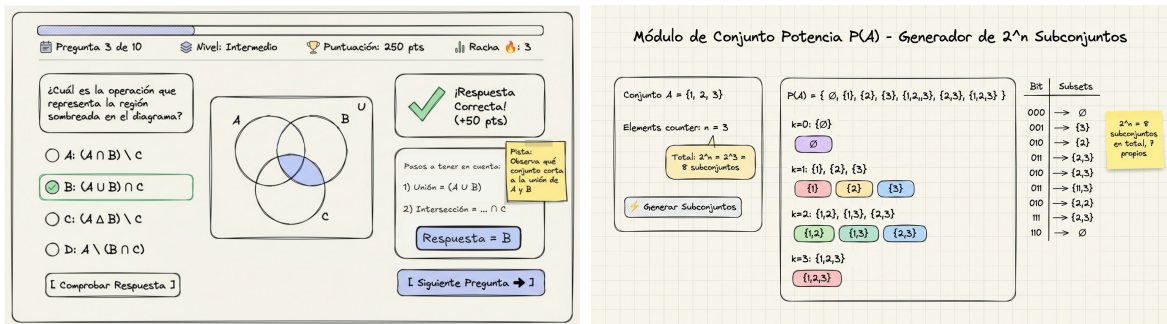


Figura 13.1: Propuestas Fernando y Mike (Linus, archivos originales)

Sprint 2 — Motores

Se implementaron los motores en TypeScript puro con pruebas iniciales: el evaluador con 15 pruebas, el solver con tipos y primera regla, el parser léxico-sintáctico de conjuntos y el híbrido de cuantificadores con cota de 2000 elementos.



Figura 13.2: Parser del equipo Sinergia, detalle (material original)

Sprint 3 — Interfaz

Se desarrollaron las vistas Vue con diagrama SVG, trazabilidad, tablas de verdad y catálogo de leyes, con componentes compartidos de botones, tarjetas e insignias y marca azul común.

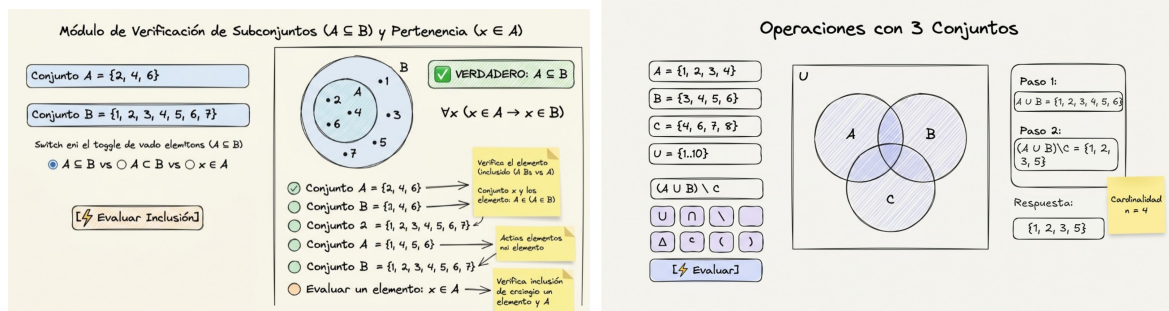


Figura 13.3: Propuestas Nio y Sergio (Linus, archivos originales)

Sprint 4 — Refinamiento y calidad

Se incorporaron mejoras solicitadas (paréntesis, cuadros estadísticos, sub-selector de potencia, corrección del complemento y del SVG, normalización de textos), se alcanzaron 189 pruebas y se documentó el despliegue.

Asistente de Inferencias Lógicas

Demostrar:

1.	$P \rightarrow Q$	
2.	$Q \rightarrow R$	
3.	P	
4.	Q	MP(1,3)
5.	R	MP(2,4)

¡Demostración completa!
La conclusión ha sido demostrada correctamente.

Historial de pasos

4	4	Q	MP (1,3)	De 1: $P \rightarrow Q$ y 3: P se obtiene Q
5	5	R	MP (2,4)	De 2: $Q \rightarrow R$ y 4: Q se obtiene R

Regla

Modus Ponens

Modus Ponens

Si $P \rightarrow Q$ y P

Entonces Q

Líneas

- ☒ 1 $P \rightarrow Q$
- ☐ 2 $Q \rightarrow R$
- ☒ 3 P
- ☐ 4 Q
- ☐ 5 R

Seleccionadas: (1,3)

Selecciona las líneas que deseas usar y luego aplica la regla.

Aplicar regla

Equivalencias

Ver equivalencias

Acciones

Reiniciar demostración

Método: Directo Líneas usadas: 5 Estado: Completado

Figura 13.4: Entorno de práctica del equipo Hijos de Linus (archivo original)

Capítulo 14

Catálogo de pruebas por archivo

Tabla 14.1: Archivos de prueba y cobertura

Archivo	Cubre
<code>evaluator.test.ts</code>	Parser, evaluador y clasificación de tablas
<code>parser.test.ts</code>	Precedencia y errores del solver
<code>solver.test.ts</code>	Diez reglas y contraejemplos
<code>astVisual.test.ts</code>	Serialización del árbol
<code>engine.test.ts</code>	Tokenizador y dominios de cuantificadores
<code>operations.test.ts</code>	Unión, intersección, potencia (14 casos)
<code>trazabilidad.test.ts</code>	Historial con 9 pruebas y 22 aserciones
<code>validator.test.ts</code>	Sanitización con 27 pruebas
<code>validator_stress.test.ts</code>	Inyecciones y anidación (15 pruebas)
<code>integracion_motor.test.ts</code>	Tubería completa sanitización-demostración
<code>ArbolAST.test.ts</code>	Render del árbol
<code>FormularioInferencia.test.ts</code>	Entrada de premisas
<code>IndicadorResultado.test.ts</code>	Estados válida/inválida
<code>PanelTrazabilidad.test.ts</code>	Pasos y exportación
<code>TraductorLenguajeNatural.test.ts</code>	Mapas de variables
<code>index.test.ts</code>	Integración de la página
<code>test_ejemplo.test.ts</code>	Ejemplo mínimo

La matriz de casos borde del validador cubre entradas vacías, caracteres prohibidos, emojis, inyecciones, operadores en múltiples notaciones, paréntesis anidados y desbalanceados, y operadores consecutivos.

Capítulo 15

Leyes formales con demostración esquemática

Idempotencia, asociativa y conmutativa se prueban por inspección de 4 filas. La distributiva requiere 8 filas. Morgan se prueba mostrando el intercambio $\wedge \leftrightarrow \vee$ fila a fila. La absorción se reduce por casos en p . La identidad se lee directo de las constantes. La expansión usa que $q \vee \neg q$ es tautología. La contraposición comparte tabla con el condicional. La exportación anida condicionales. La definición traduce $\rightarrow$ y $\leftrightarrow$ a $\neg$, $\vee$ y $\wedge$.

```
1)  $(\exists x)[\sim Px \leftrightarrow (Qx \vee \sim Rx)]$   
  
=  $(\exists x)[(\sim Px \rightarrow (Qx \vee \sim Rx)) \wedge ((Qx \vee \sim Rx) \rightarrow \sim Px)] \dots$   
Definición de Bicondicional  
  
=  $(\exists x)[(Px \vee (Qx \vee \sim Rx)) \wedge ((Qx \vee \sim Rx) \rightarrow \sim Px)] \dots$  Definición  
de Implicación y Doble Negación  
  
=  $(\exists x)[(Px \vee (Qx \vee \sim Rx)) \wedge \sim((Qx \vee \sim Rx) \vee \sim Px)] \dots$  Definición  
de Implicación y Doble Negación  
  
=  $(\exists x)[(Px \vee (Qx \vee \sim Rx)) \wedge [\sim(Qx \vee \sim Rx) \wedge Px]] \dots$  Ley de De  
Morgan y Doble Negación  
  
=  $(\exists x)[(Px \vee (Qx \vee \sim Rx)) \wedge [\sim Qx \wedge Rx \wedge Px]] \dots$  Ley de De  
Morgan y Doble Negación  
  
=  $(\exists x)(Px \vee (Qx \vee \sim Rx)) \wedge (\exists x)[\sim Qx \wedge Rx \wedge Px] \dots$   
Distribución de Cuantificadores  
  
=  $(\exists x)Px \vee (\exists x)(Qx \vee \sim Rx) \wedge (\exists x)[\sim Qx \wedge Rx \wedge Px] \dots$   
Distribución de Cuantificadores
```

Figura 15.1: Demostración paso a paso del motor de leyes (Modus Innova, material original)

Capítulo 16

Módulos de validación, trazabilidad y transcripción

El validador normaliza más de 15 notaciones ($\rightarrow$, $\wedge$, $\rightarrow$, $\&\&$, palabras) hacia claves canónicas, previene inyecciones y valida paréntesis para formularios reactivos. La trazabilidad acumula pasos en orden con instantáneas de estado y reexporta tipos y funciones. La transcripción mapea cada regla a nombre, alias y descripción general, arma oraciones citando líneas y renderiza el árbol a texto legible.

Capítulo 17

Páginas Vue y sistema de contenido

Las nueve vistas orquestan entradas reactivas, motores y contenido sin contener lógica: Tablas detecta variables en vivo; Conjuntos computa regiones y colores del Venn; Inferencias procesa en asíncrono con historial local; Cuantificadores evalúa en dual y resuelve leyes; Inicio, Aprender, Leyes y Progreso componen hero, recorrido de cuatro pasos, búsqueda y gráfico de precisión. El contenido visible proviene de claves por módulo con interruptor de modo literal.



Figura 17.1: Propiedades computadas de la vista (material original Sinergia)

Capítulo 18

Integración Vue y contenido externalizado

Cada página orquesta entradas reactivas, motores y contenido: la entrada se parsea, se detectan variables en vivo, se generan filas y se clasifica; los errores de parseo se muestran sin romper la vista. El contenido visible proviene de claves por módulo con interruptor de modo literal que elige símbolos o palabras. La correspondencia pantalla-contenido se resume en la Tabla 18.1.

Tabla 18.1: Trazabilidad pantalla-contenido (selección)

Elemento	Clave
Tablas: variables y conectores	<code>tablas.info</code>
Cuantificadores: dominio y predicado	<code>cuantificadores.dominio</code> , <code>.predicado</code>
Inferencias: trazabilidad	<code>inferencias.trazabilidad</code>
Conjuntos: operaciones y Venn	<code>conjuntos.operaciones</code> , <code>.diagrama</code>

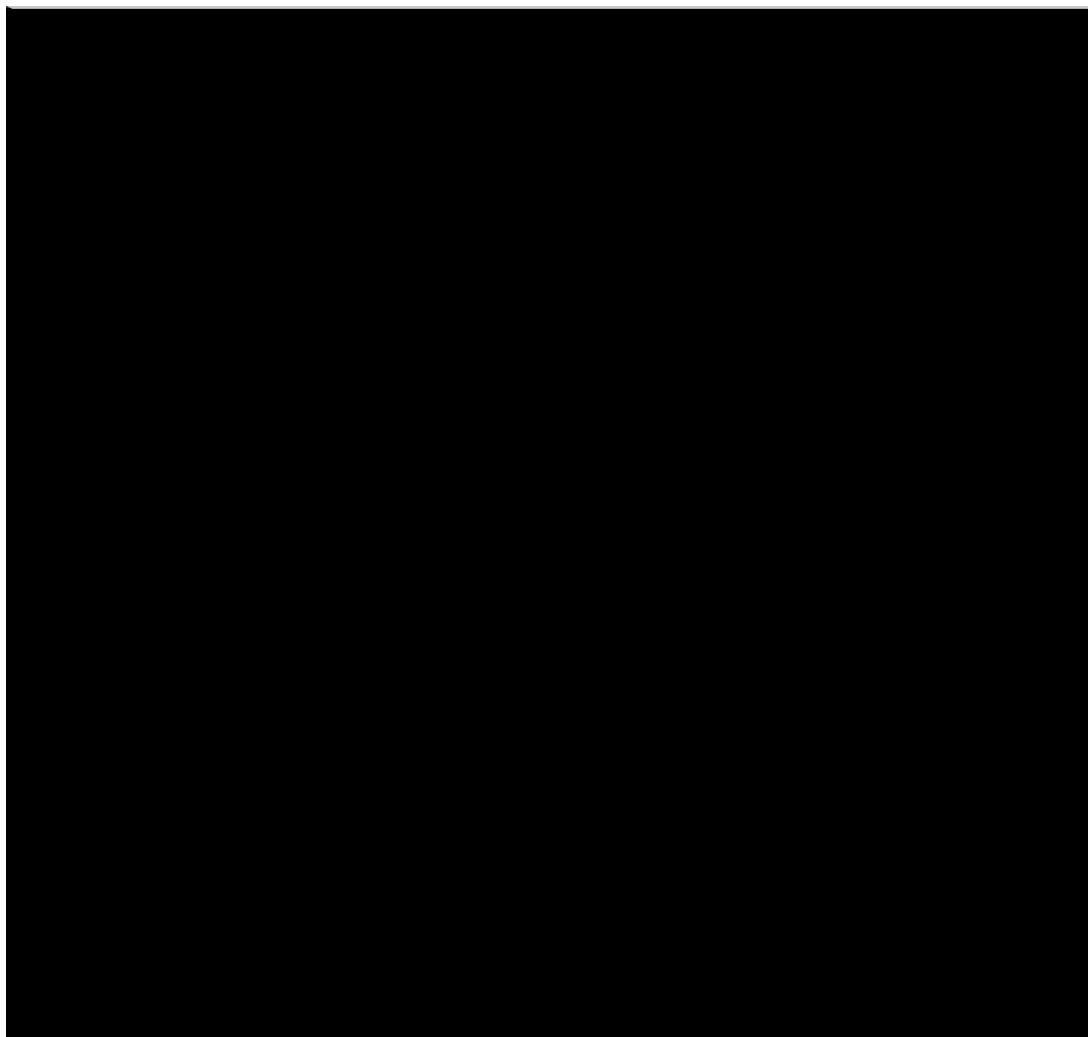


Figura 18.1: Estado reactivo de la vista de tablas (material original Sinergia)

Capítulo 19

Pruebas, calidad y despliegue

La suite suma 189 pruebas en 17 archivos que validan parsers, evaluadores, leyes y componentes, con verificación de tipos estricta y compilación de producción. El despliegue publica la compilación en GitHub Pages con redirección para rutas y vistas previas por Pull Request.



Figura 19.1: Cronograma de sprints (material original Sinergia)

Capítulo 20

Evolución por sprints

Propuesta con wireframes y decisión de Vue 3; motores con pruebas iniciales; interfaz con componentes compartidos y diagramas; calidad con 189 pruebas y documentación. Los bocetos originales se conservan como evidencia.

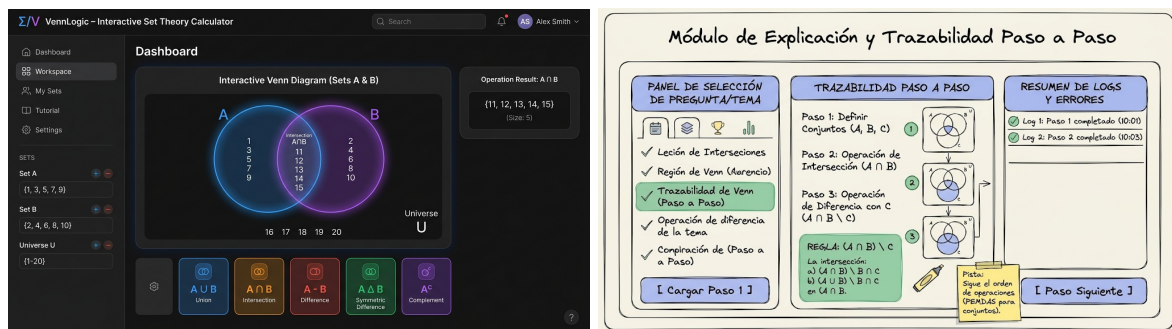


Figura 20.1: Bocetos del equipo Linus (archivos originales)

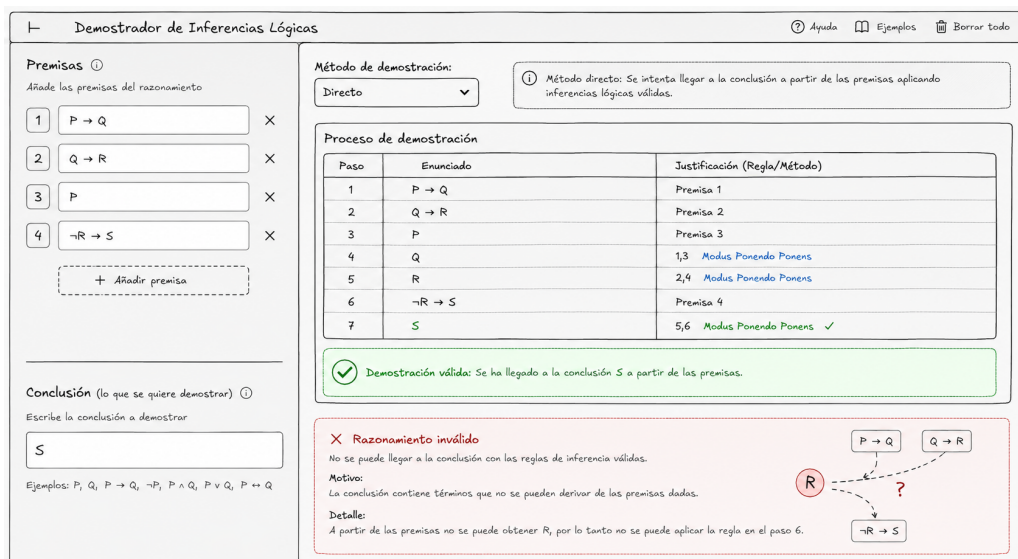


Figura 20.2: Interfaz del equipo Hijos de Linus (archivo original)

Capítulo 21

Conclusiones

La separación entre motores y vistas permitió colaborar sin colisiones y editar contenido docente sin código. Los límites explícitos (2^v filas, 2000 elementos de dominio, 12 iteraciones de encadenamiento) muestran conciencia de complejidad. El trabajo futuro incluye exportación de tablas, calificación y lógica de primer orden completa.

Capítulo 22

Anexo: bitácoras de sprint (transcripción fiel formateada)

Este anexo transcribe fielmente las bitácoras registradas por los equipos en `extttTAREAS/`, convertidas de Markdown a $\text{\LaTeX}$ con ortografía corregida y terminología unificada ($D \subset \mathbb{Z}$, conectivos $\neg$, $\wedge$, $\vee$, $\circ$, $\leftrightarrow$). Constituyen la evidencia de trabajo por sprints.

Propuesta Sinergia

Fuente original: `../TAREAS/01_Propuesta/sinergia/propuesta.md`

LogiLearn

Equipo: Sinergia

Integrantes:

- Julio Miguel Velarde Avila
- Segundo Jesús Nuñez Esquen
- Aldair Wilfredo Querevalu Esquen
- Luis Smith Atencio Fiestas
- Alexa Jhosely Benavides Irigoin (líder)

Tema: Tablas de verdad y leyes lógicas

1. Problema a Resolver

El aprendizaje de la lógica simbólica puede resultar complicado para los estudiantes debido a la dificultad para comprender y aplicar conceptos como proposiciones, conectores lógicos, tablas de verdad y leyes lógicas. Además, el aprendizaje exclusivamente teórico puede dificultar la práctica y la identificación de errores durante la resolución de ejercicios.

2. Funcionalidad Propuesta

Se propone desarrollar una plataforma web interactiva que facilite el aprendizaje de la lógica simbólica mediante explicaciones, ejemplos y ejercicios prácticos. La herramienta permitirá al estudiante practicar los principales conceptos de lógica simbólica y recibir una retroalimentación que facilite su aprendizaje. LogiLearn es una plataforma web interactiva para aprender lógica proposicional de forma visual. Incluye:

- **Página de inicio:** presenta la herramienta y sus módulos principales (Tablas de verdad, Leyes lógicas, Ejercicios prácticos y Tu progreso), con acceso mediante inicio de sesión.
- **Módulo de Tablas de Verdad:** permite ingresar una proposición lógica (usando variables p, q, r y operadores como Y, O, NO), generar automáticamente su tabla de verdad y visualizar el resultado de la expresión evaluada.
- **Módulo de Leyes Lógicas:** muestra de forma didáctica las distintas leyes lógicas (De Morgan, Distributiva, entre otras) organizadas en tarjetas visuales, facilitando su comprensión y repaso.
- **Seguimiento de progreso:** el sistema permite al usuario continuar su aprendizaje desde donde lo dejó, guardando su avance en los módulos.

3. Boceto Visual (Wireframe)

Diseño elaborado en Figma: Ver diseño en Figma (<https://www.figma.com/design/MRiWbcWmQAohkH3CqKPCInterfaz-Web---Inicio?node-id=81-3&t=iMJkIZVvPtwiNntF-1>)

```
<img width='1446' height='13186' alt='Interfaz Web - Inicio (1)' src='https://github.com/user-attachments/assets/afc3e676-fb55-4c4c-a768-c5f18a70b53f' />
```

Propuesta Hijos de Linus

Fuente original: ../TAREAS/01_Propuesta/hijos-de-linus/propuesta.md

Asistente e Intérprete de Inferencias Lógicas

Equipo: Los Hijos de Linus **Integrantes:** Espinoza Arom, Centurión Alex, Mio Arnold, Morocho Juan, Altamirano Renatto **Tema:** Inferencias Lógicas

1. Problema a Resolver

El aprendizaje de las inferencias lógicas en estudiantes universitarios presenta dos dificultades principales: 1. **Dificultad en la resolución asistida y verificación:** A los estudiantes les cuesta comprobar de forma rápida si un razonamiento lógico es válido o inválido, así como visualizar la demostración formal mediante método directo o indirecto. 2. **Dificultad en el aprendizaje interactivo paso a paso:** Muchos estudiantes tienen problemas para identificar y aplicar correctamente las reglas de inferencia por sí mismos en el desarrollo de una prueba, cometiendo errores de deducción sin recibir retroalimentación inmediata sobre qué falla en su razonamiento.

2. Funcionalidad Propuesta

La herramienta consistirá en un sistema web interactivo compuesto por **dos interfases (módulos) principales:**

2.1. Interfaz de Calcular y Resolver (Resolución Automática)

- **Ingreso de Datos:** El usuario ingresa un conjunto de premisas y la conclusión que se desea demostrar.
- **Selección de Método:** Permite elegir entre el **Método Directo** o el **Método Indirecto** (Reducción al absurdo).

- **Motor de Resolución:** El sistema reescribe las premisas utilizando equivalencias lógicas según sea conveniente y genera la demostración completa del razonamiento de manera automática.

2.2. Interfaz de Practicar Ejercicios (Resolución Guiada e Interactiva)

- **Entorno de Práctica:** El estudiante inicia con las premisas y la conclusión del ejercicio a resolver.
- **Aplicación Manual de Reglas:** El usuario selecciona manualmente una regla de inferencia y especifica sobre qué líneas de premisas desea aplicarla.
- **Validación en Tiempo Real:**
 - Si la aplicación es **correcta**, el sistema agrega automáticamente la nueva proposición derivada y habilita el siguiente paso.
 - Si la aplicación es **incorrecta**, la plataforma muestra un mensaje de retroalimentación indicando el error cometido.
- **Objetivo:** Acompañar el proceso de aprendizaje permitiendo al estudiante construir la demostración paso a paso hasta llegar a la conclusión.

3. Boceto Visual (Wireframe)

Módulo 1: Interfaz de Calcular y Resolver !Interfaz de Calcular y Resolver ([assets/propuesta-arnold.png](#))

Módulo 2: Interfaz de Practicar Ejercicios !Interfaz de Practicar Ejercicios ([assets/propuesta-morocho.jpeg](#))

Observación

La herramienta busca integrar tanto un componente utilitario de resolución automatizada como un componente pedagógico interactivo. Para un desarrollo eficiente, la plataforma dispondrá de un módulo centralizado de equivalencias y reglas de inferencia lógicas reutilizable en ambos sistemas.

Propuesta Linus

Fuente original: ../TAREAS/01_Propuesta/linus/propuesta.md

Asistente e Intérprete de Teoría de Conjuntos y Diagramas de Venn

Equipo: Equipo Linus **Integrantes:** Jordy (Sublíder), Junior, Mike, Sergio, Fernando, Alejandro **Tema:** Teoría de Conjuntos y Diagramas

1. Problema a Resolver

El aprendizaje de la Teoría de Conjuntos y los Diagramas de Venn en estudiantes universitarios presenta dos dificultades principales: 1. **Dificultad en la visualización espacial y cálculo de operaciones:** A los estudiantes les cuesta comprobar de forma rápida si una operación entre dos o tres conjuntos produce el resultado correcto y qué elementos exactos pertenecen a cada región del diagrama. 2. **Dificultad en el cálculo del Conjunto Potencia y relaciones de pertenencia/inclusión:** Muchos estudiantes confunden pertenencia con inclusión y tienen problemas para listar manualmente todos los subconjuntos.

2. Funcionalidad Propuesta

La herramienta consistirá en un sistema web interactivo compuesto por **dos módulos principales**:

2.1. Calculadora de Conjuntos y Diagrama de Venn

- El usuario ingresa un Universo U y los conjuntos A , B y opcionalmente C .
- Permite elegir entre Unión, Intersección, Diferencia, Diferencia Simétrica, Complemento o Conjunto Potencia.
- Genera el Diagrama de Venn iluminando en tiempo real la región correspondiente.

2.2. Evaluador de Propiedades y Práctica Guiada

- Comprueba automáticamente si $A \subseteq B$, si $A = B$, o si son disjuntos.
 - Genera todos los subconjuntos del Conjunto Potencia con explicación paso a paso.
 - Permite a los estudiantes validar sus ejercicios de forma inmediata.
-

3. Boceto Visual (Wireframe)

Módulo de Calculadora e Interfaz de Venn (Jordy) !Boceto de Jordy ([assets/propuesta-jordy.jpg](#))

Módulo de Verificación de Subconjuntos (Junior) !Boceto de Junior ([assets/propuesta-junior.jpg](#))

Módulo de Conjunto Potencia (Mike) !Boceto de Mike ([assets/propuesta-mike.jpg](#))

Módulo de Operaciones con 3 Conjuntos (Sergio) !Boceto de Sergio ([assets/propuesta-sergio.jpg](#))

Módulo de Ejercicios Didácticos (Fernando) !Boceto de Fernando ([assets/propuesta-fernando.jpg](#))

Módulo de Explicación Paso a Paso (Alejandro) !Boceto de Alejandro ([assets/propuesta-alejandro.jpg](#))

Observación

La herramienta busca integrar un módulo de resolución y graficación de Venn con un componente didáctico de verificación de propiedades para fortalecer los conceptos de Teoría de Conjuntos.

Propuesta Modus Innova

Fuente original: ../TAREAS/01_Propuesta/modus-innova/propuesta.md

Validador y Tutor Interactivo de Cuantificadores y Silogismos Lógicos (QuantifiWeb)

Equipo: Equipo 4 — Modus Innova (QuantifiTech) **Integrantes:** Cristian (Sublíder), Danuska, Marlon, Guillermo, Noemí **Tema Asignado:** Cuantificadores Lógicos, Lógica de Predicados y Silogismos / Inferencias

1. Descripción del Problema

En el aprendizaje de la Lógica Simbólica y Predicados, a los estudiantes universitarios les resulta especialmente difícil enfrentarse a dos conceptos fundamentales:

1. **Cuantificadores Lógicos ($\forall$ Universal y $\exists$ Existencial):** A los alumnos les cuesta evaluar el valor de verdad (Verdadero o Falso) de proposiciones cuantificadas sobre un **Dominio de Discurso** finito (ej. $D = \{1, 2, 3, 4, 5\}$). Además, suelen confundirse al aplicar las **Leyes de Negación de Cuantificadores (De Morgan)**:

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x)$$

$$\neg(\exists x P(x)) \equiv \forall x \neg P(x)$$

y en comprender cómo un solo elemento en el dominio puede actuar como contraejemplo que invalide una afirmación universal.

2. **Inferencias Lógicas y Silogismos:** Les resulta difícil identificar si un argumento (compuesto por varias premisas y una conclusión) es **formalmente válido o es una falacia**. Asimismo, al intentar resolver ejercicios manuscritos, frecuentemente se confunden sobre **qué Regla de Inferencia aplicar** (*Modus Ponens*, *Modus Tollens*, *Silogismo Hipotético*, *Silogismo Disyuntivo*) y cómo justificar cada paso formal para llegar a la conclusión.

2. ¿Qué hará nuestro componente?

Nuestra herramienta web **QuantifiWeb** será una interfaz visual interactiva que integrará dos módulos funcionales explicativos y aplicativos en tiempo real:

1. Módulo de Cuantificadores Lógicos (Lógica de Predicados):

- **Definición de Dominio y Predicados (Inputs):** El usuario ingresa un conjunto universo D (ej. 1, 2, 3, 4, 5) y la condición de su predicado $P(x)$ (ej. 'x es par').
- **Evaluación en Dominio con Trazabilidad:** Al presionar 'Evaluar Cuantificador', el sistema evalúa la fórmula elemento por elemento dentro del dominio D , mostrando la tabla de trazabilidad ($P(1) = F$, $P(2) = V$, etc.), el resultado global (**VERDADERO** o **FALSO**) y el contraejemplo o caso de éxito.

- **Asistente de Negación (De Morgan):** Botón '**Negar Expresión**' que calcula automáticamente la proposición equivalente con el cuantificador opuesto y su negación interna.

2. Módulo de Inferencias Lógicas y Silogismos:

- **Ingreso del Argumento (Inputs):** El usuario verá dos cajas de texto principales para ingresar la **Premisa 1** (ej. $p \rightarrow q$) y la **Premisa 2** (ej. p), además de una caja de texto para la **Conclusión** (ej. q).
- **Barra de Teclado Rápido de Símbolos:** Contará con una barra de **botones rápidos de símbolos lógicos** ($, , , , , ,$) para facilitar la escritura sin cometer errores de tipeo.
- **Verificación e Interacción Paso a Paso:** Al hacer clic en el botón '**Validar Inferencia**':
- **Si la inferencia es VÁLIDA:** El panel se iluminará en verde, mostrará un mensaje destacado como **Argumento Valido!** y desplegará la explicación paso a paso indicando la regla exacta utilizada (ej. '*Se aplicó la regla Modus Ponens [MP] a partir de la Premisa 1 y Premisa 2*').
- **Si la inferencia es INVÁLIDA (Falacia):** El panel se pondrá de color rojo alertando **Argumento Invalido (Falacia)**, y mostrará un ejemplo de la vida real (contraejemplo) explicando por qué las premisas no garantizan esa conclusión.

3. Modo Didáctico de Práctica ('Cargar Ejemplo'):

- Incluirá un botón de '**Cargar Ejemplo**' para que los alumnos que no tengan ejercicios a la mano puedan probar casos preescritos de Modus Ponens, Modus Tollens, Silogismos y Cuantificadores.

3. Boceto Visual (Wireframe)

A continuación se presenta la distribución estructurada de pantallas, componentes, entradas y paneles de resultado de la herramienta:

```

1  +-----+
2  | LOGICA SIMBOLICA GLOBAL | Equipo 3: Modus Innova -- QuantifiWeb |
3  +-----+
4  | [ 1. APRENDE (Teoria Interactiva) ] [ 2. PRACTICA Y COMPRUEBA (Evaluador) ] |
5  +-----+
6  |
7  | SECCION SELECCIONADA: PRACTICA Y COMPRUEBA |
8  |
9  | |
10 | 1. CONFIGURACION DE CUANTIFICADORES Y DOMINIO D |
11 | Dominio (D): [ 1, 2, 3, 4, 5 ] |
12 | Predicado P(x): [ x es numero par ] |
13 | Formula: [ x P(x) ] |
14 | Simbolos: [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] |
15 |
16 | [ Evaluar Cuantificador ] [ Negar Expresion ] [ Cargar Ejemplo ] |
17 |
18 |
19 |
20 | 2. EVALUACION Y TRAZABILIDAD PASO A PASO |
21 | STATUS: PROPOSICION FALSA |
22 | Detalle por Elemento en D = {1, 2, 3, 4, 5}: |
23 | - Para x = 1: P(1) es Falso (1 no es par) [CONTRAEJEMPLO HALLADO] |
24 | - Para x = 2: P(2) es Verdadero (2 es par) |
25 | - Para x = 3: P(3) es Falso (3 no es par) |
26 | - Para x = 4: P(4) es Verdadero (4 es par) |
27 | Explicacion: El cuantificador exige cumplimiento en el 100% del dominio. |
28 | Equivalente Negado (De Morgan): x P(x) "Existe al menos un x que NO es par" |
29 |
30 |

```

```

31 | |
32 | 3. INGRESO Y VALIDADOR DE INFERENCIAS LOGICAS (SILOGISMOS) |
33 | Premisa 1 (P1): [ p q ] |
34 | Premisa 2 (P2): [ p ] |
35 | Conclusion (): [ q ] |
36 | Simbolos: [ ] [ ] [ ] [ ] [ ] |
37 | |
38 | [ Validar Inferencia ] [ Cargar Ejemplo ] |
39 | |
40 | |
41 | |
42 | 4. PANEL DE RESULTADO Y DEMOSTRACION DE LA INFERENCIA |
43 | STATUS: ARGUMENTO VALIDO! |
44 | Explicacion Paso a Paso: |
45 | - Paso 1: Tienes la implicacion 'p q' (Premisa 1). |
46 | - Paso 2: Ocurre el antecedente 'p' (Premisa 2). |
47 | - Regla Aplicada: MODUS PONENDO PONENS (MP). |
48 | - Conclusion: Se deduce obligatoriamente 'q'. |
49 | |
50 | |
51 | +-----+

```

Avance Sprint 2 Hijos de Linus

Fuente original: ../TAREAS/02_Motor/hijos-de-linus/avance.md

Avances del Sprint 2 - Equipo: Hijos de Linus

Integrante: Arom

Módulo: Motor Lógico (Solver)

Completado (15 de Agosto)

■ Definición de Tipos y AST:

- Se establecieron los operadores lógicos en español (Y, O, O_EXCLUSIVA, NO, ENTONCES, SI_Y_SOLO_SI, NI, INCOMPATIBLE).
- Se definieron los nodos del Árbol de Sintaxis Abstracta (AST) en `src/lib/solver/types.ts`.
- Se crearon los tipos `Inferencia` y `Equivalencia` para representar las reglas lógicas (Modus Ponens, Modus Tollens, Silogismo, De Morgan, etc.) y se mapearon sus alias populares.

■ Base del Evaluador (`solver.ts`):

- Se creó la función principal `demostrarConclusion` que recibe un arreglo de premisas y la conclusión deseada.
- Se implementó el primer evaluador `aplicarModusPonendoPonens` como prueba de concepto para el 'pattern matching'.
- Se creó la función `sonNodosIguales` para verificar la similitud estructural de dos ramas lógicas (esencial para evaluar equivalencias y silogismos).

■ Pruebas y Verificación:

- Se creó la suite de pruebas automatizadas en `src/lib/solver/solver.test.ts`.
- El código base cumple estrictamente el tipado estricto (0 errores en `vue-tsc`) y 100% de cobertura en las pruebas iniciales de Vitest.

■ Analizador Sintáctico (Parser) (Paso 2):

- Se implementó el analizador léxico (`tokenizar`) para separar correctamente variables, operadores y paréntesis respetando espacios.
- Se construyó el algoritmo de parseo (`construirAST`) mediante un analizador de descenso recursivo, respetando firmemente la jerarquía y precedencia de operadores lógicos (donde 'NO' tiene mayor peso que 'Y', seguido de 'O', y por último las condicionales/bicondicionales).
- Se superaron las pruebas unitarias exhaustivas para expresiones anidadas, precedencia correcta y validación de errores sintácticos (`parser.test.ts`).

Pendiente y Notas para el Futuro

- **Expansión de Reglas (Módulo de Arom):** Seguir agregando las funciones unitarias para detectar y aplicar `MODUS_TOLLENDO_TOLLENS`, `SILOGISMO_HIPOTETICO`, `DE_MORGAN`, etc., basándose en la plantilla estructural de `aplicarModusPonendoPonens`.
- **Algoritmo de Búsqueda:** Mejorar el interior de `demostrarConclusion` para que recorra cíclicamente el arreglo de premisas intentando derivar proposiciones nuevas usando encadenamiento hacia adelante (forward-chaining) hasta dar con la conclusión deseada.

Integrantes: Morocho y Alex

Módulo: Trazabilidad y Explicación (Didáctico)

Completado (16 de Agosto)

- **Diseño de Estructura de Paso (`types.ts`):**
 - Se definió la interfaz `PasoTrazabilidad` con los campos: `numeroPaso`, `operacion`, `regla`, `alias`, `explicacion`, `expresionSimbolica`, `lineasBase`, `esConclusion` y `estadoActual`.
 - Se definió `ResultadoTrazabilidad` como contenedor completo con `esValido`, `pasos`, `conclusion` y `totalPasos`, listo para consumir por la UI en el próximo sprint.
- **Contenedor del Historial (`historial.ts`):**
 - Se creó `crearHistorial()` que retorna un array vacío `[]` como punto de partida.
 - Se implementó `registrarPaso()` que recibe un paso del solver, lo traduce a `PasoTrazabilidad` usando `generarPasoEnriquecido` de Mio, genera un snapshot del `estadoActual` con todas las líneas disponibles, y lo añade al array con `.push()`.
 - Se implementaron funciones auxiliares `obtenerPaso(numero)` y `obtenerConclusiones()` para consultas sobre el historial.
- **Integración con el Solver (`construirTrazabilidad()`):**
 - Se creó la función principal que recibe `premisas[] + ResultadoDemostracion` del solver y produce un `ResultadoTrazabilidad` completo.
 - Recorre los pasos EN ORDEN acumulando `pasosPrevios` para resolver referencias cruzadas de líneas, consumiendo el módulo de transcripción de Mio (`generarPasoEnriquecido`, `renderizarNodo`).
 - Genera la conclusión final en lenguaje natural según `resultado.esValido`.

- **Punto de Entrada (`index.ts`):**

- Se creó barrel file que re-exporta tipos y funciones del módulo.

- **Pruebas y Verificación:**

- Se creó suite de pruebas en `trazabilidad.test.ts` con 9 tests (22 assertions) cubriendo: creación de historial vacío, registro de pasos con datos completos, acumulación de múltiples pasos, generación de `estadoActual`, proceso completo de `construirTrazabilidad`, y funciones auxiliares de consulta.
- Código cumple estrictamente el tipado (0 errores en `vue-tsc`) y 100 % de cobertura en pruebas.

- **Corrección de Bug:**

- Se corrigió importación rota en `src/lib/transcription/astRenderer.ts`: apuntaba a `'./types'` (inexistente) en vez de `'../solver/types'`.

Integrante: Mio

Módulo: Transcripción

Completado (16 de Agosto)

- **Diccionario de Traducciones:**

- Se creó `translations.ts`, indexado por `ReglaLogica` (el tipo combinado `Inferencia | Equivalencia` que definió Arom en `types.ts`), usando `Record<ReglaLogica, InfoRegla>` para forzar en tiempo de compilación que **las 16 reglas** (8 de inferencia + 8 de equivalencia) tengan traducción.
- Cada regla mapea a un objeto `InfoRegla` con `nombre` (nombre para mostrar), `alias` opcional (ej. 'MPP') y `descripcion` (qué hace la regla en general, en español llano).

- **Generador de descripciones:**

- Se crearon las funciones:
- `resolverLinea(numeroLinea, premisas, pasosPrevios)`: traduce un número de `lineasInvolucradas` a la expresión (`NodoExpresion`) real a la que apunta, siguiendo la convención de numeración de Arom (líneas 1..N = premisas originales; líneas N+1 en adelante = pasos ya demostrados, en orden).
- `generarDescripcionPaso(paso, premisas, pasosPrevios)`: arma la oración completa en español citando qué líneas se usaron (con su expresión renderizada), qué regla se aplicó (nombre + alias + descripción general) y qué expresión se obtuvo, agregando una frase de cierre si `paso.esConclusion` es verdadero.
- Se separó el renderizado del AST a texto legible (`(P ENTONCES Q)`, `(NO P)`, etc.) en una función auxiliar recursiva (`renderizarNodo`), reutilizable en cualquier parte que necesite mostrar una expresión, no solo dentro de una explicación.
- Se agregó `generarPasoEnriquecido`, que devuelve los mismos datos pero 'desarmados' en campos separados (regla, alias, expresión resultante, descripción, `esConclusion`), por si la UI necesita mostrarlos en columnas en vez de un párrafo corrido.

- **Integración Trazable:**

- Se creó `index.ts` como punto de entrada único del módulo.
- `traducirHistorial(premisas, resultado)` recorre `resultado.pasos en orden` (no en paralelo), manteniendo un acumulador de pasos ya procesados — necesario porque un paso puede depender de una línea generada por un paso anterior, no solo de las premisas originales.
- Cada paso traducido conserva su número de línea calculado (`premisas.length + indice + 1`) y su `lineasInvolucradas` original, para poder cruzarlo 1 a 1 con el historial crudo de Arom si Morocho o Alex lo necesitan.
- `explicarDemostracion(premisas, resultado)` envuelve lo anterior y agrega una conclusión final en lenguaje natural según `resultado.esValido`.
- Se validó la integración corriendo `ejemplo.ts` contra el `demostrarConclusion` real de Arom (mismo caso que su `solver.test.ts`, Modus Ponendo Ponens con P ENTONCES Q y P), confirmando compilación en modo estricto (0 errores) y salida correcta en español.

Integrante: Renato

Módulo: Validaciones, Sanitización, Pruebas y Cobertura (Control de Calidad)

Completado (17 de Agosto)

- **Matriz de Casos Borde (`casos_de_prueba.md`):**
 - Se redactó una matriz exhaustiva de 25 casos de prueba y escenarios límite categorizados (entradas vacías, caracteres prohibidos/especiales, emojis, inyecciones de código/HTML, operadores matemáticos y lógicos en múltiples notaciones, variables compuestas con índices, paréntesis anidados, desbalanceados o vacíos, operadores consecutivos e incompletos, premisas redundantes, etc.).
- **Módulo de Sanitización y Validación (`src/lib/validator/`):**
 - Se diseñaron las interfaces de datos en `types.ts` (`ResultadoValidacion`, `ResultadoValidacionConjunto`, `OpcionesSanitizacion`).
 - Se implementó `sanitizarEntrada(texto)` en `validator.ts`, que normaliza espacios redundantes, unifica notaciones lógicas universales (`->`, `,`, `~`, `&`, `|`, etc.) hacia las palabras clave estándar en mayúsculas del motor (`ENTONCES`, `Y`, `O`, `NO`, `SI_Y_SOLO_SI`, `O_EXCLUSIVA`, `NI`, `INCOMPATIBLE`), previene inyecciones maliciosas y valida la consistencia léxica y de paréntesis.
 - Se implementó `validarExpresion(texto)` como función segura orientada a formularios reactivos en Vue (Sprint 3), retornando objetos de estado sin arrojar excepciones no controladas.
 - Se implementó `validarPremisasYConclusion(premisas, conclusion)` para el procesamiento de lotes con deduplicación automática y acumulación de diagnósticos de error.
- Se creó el barrel file `index.ts` y la guía de uso `README.md` para el equipo.
- **Pruebas Automatizadas y Cobertura:**
 - Se desarrolló la suite unitaria `validator.test.ts` con 27 tests detallados cubriendo todos los casos de la matriz.
 - Se desarrolló la suite de integración end-to-end `integracion_motor.test.ts` que valida el pipeline completo de todos los módulos (`Sanitizacion -> Parser -> Solver -> Trazabilidad -> Transcripcion`).

- Se desarrolló la suite de pruebas avanzadas y estrés `validator_stress.test.ts` (15 tests adicionales) para comprobar resiliencia ante inyecciones maliciosas, fuzzer de expresiones corruptas, 10 niveles de anidación y deduplicación avanzada.
- Todas las suites del proyecto (7 archivos de pruebas y 67 tests) pasan al 100 % en Vitest sin fallos ni regresiones.
- Se verificó la conformidad estricta de TypeScript con `vue-tsc -noEmit` (0 errores de tipado) y compilación limpia en producción con `npm run build`.

Casos de prueba Hijos de Linus

Fuente original: ../TAREAS/02_Motor/hijos-de-linus/casos_de_prueba.md

Matriz de Casos Borde y Pruebas (Módulo de Validación)

Autor: Renato (Hijos de Linus) **Objetivo:** Proteger el motor lógico y el pipeline de inferencia simbólica ante cualquier intento de ruptura, entradas maliciosas, caracteres no permitidos, errores de sintaxis y variaciones en la notación que los usuarios puedan ingresar en la interfaz gráfica o consola.

1. Clasificación y Matriz de Casos

ID — Categoría — Entrada de Usuario (Input) — Comportamiento Esperado / Sanitización — Justificación Técnica

CB-01 — Vacío / Nulo — " (cadena vacía) — Lanzar error: *'La entrada no puede estar vacía'* — Evita procesar tokens nulos en el AST Parser.

CB-02 — Espacios en blanco — ' \t \n ' — Lanzar error: *'La entrada no puede contener solo espacios'* — Limpieza previa de espacios redundantes.

CB-03 — Espaciado irregular — ' P ENTONCES Q ' — Sanitizar a 'P ENTONCES Q' — Normalización por regex de espacios múltiples a uno solo.

CB-04 — Minúsculas / Mayúsculas — 'p entonces q' — Sanitizar a 'P ENTONCES Q' — Estandariza variables y operadores en mayúsculas.

CB-05 — Operador Flecha (->, =>,) — 'p ->q' / 'p =>q' / 'p q' — Sanitizar a 'P ENTONCES Q' — Soporte intuitivo para notaciones simbólicas populares.

CB-06 — Operador Bicondicional (<->, <=>,) — 'p <->q' / 'p <=>q' / 'p q' — Sanitizar a 'P SI_Y_SOLO_SI Q' — Soporte de doble flecha y símbolo Unicode.

CB-07 — Conjunción (^, &, &&,) — 'p ^ q' / 'p & q' / 'p && q' / 'p q' — Sanitizar a 'P Y Q' — Mapeo de conjunción estándar y de programación.

CB-08 — Disyunción (v, V, \||, \|\|,) — 'p v q' / 'p \|| q' / 'p q' — Sanitizar a 'P O Q' — Diferenciación contextual de la letra 'v' como 'O'.

CB-09 — Negación (~, !,) — '~p' / '!p' / 'p' / 'NO p' — Sanitizar a 'NO P' — Normalización de prefijos unarios de negación.

CB-10 — Disyunción Exclusiva (, ^, , XOR) — 'p q' / 'p XOR q' / 'p q' — Sanitizar a 'P O_EXCLUSIVA Q' — Compatibilidad con notación lógica avanzada.

CB-11 — NOR / Barra de Nicod (, NI,) — 'p q' / 'p NI q' — Sanitizar a 'P NI Q' — Operador de incompatibilidad conjunta.

CB-12 — NAND / Barra de Sheffer (, , NAND) — 'p q' / 'p NAND q' — Sanitizar a 'P INCOMPATIBLE Q' — Operador de negación alterna.

CB-13 — Paréntesis pegados — 'P ENTONCES(Q O R)' — Sanitizar a 'P ENTONCES (Q O R)' — Aislamiento léxico de paréntesis.

CB-14 — Paréntesis desbalanceados (Apertura sin cierre) — '(P ENTONCES Q' — Lanzar error: *'Paréntesis desbalanceados: falta cerrar ')''* — Validación sintáctica previa al parser.

CB-15 — Paréntesis desbalanceados (Cierre sin apertura) — 'P ENTONCES Q)' — Lanzar error: *'Paréntesis desbalanceados: ')' inesperado'* — Validación de orden de cierre de paréntesis.

CB-16 — Paréntesis vacíos — 'P ENTONCES ()' — Lanzar error: *'Expresión vacía dentro de paréntesis'* — Evita nodos hijos indefinidos en el AST.

CB-17 — Caracteres prohibidos / Emojis — 'P Q' / 'P @ Q' / 'P \$ Q' — Lanzar error:

'Caracteres inválidos o no reconocidos: [, @, \$]' — Bloquea inyecciones y caracteres extraños. **CB-18** — Inyección de código / Tags HTML — '`<script>alert(1)</script>`' — Lanzar error: 'Caracteres inválidos detectados: [<, >, /]' — Previene XSS y problemas de renderizado en Vue. **CB-19** — Variables con subíndices — 'P1 ENTONCES P2' / 'p_1 Y p_2' — Sanitizar a 'P1 ENTONCES P2' / 'P_1 Y P_2' — Soporte a proposiciones indexadas (P1, Q2, R_1). **CB-20** — Operadores consecutivos inválidos — 'P Y Y Q' / 'P ENTONCES O Q' — Lanzar error: 'Operadores lógicos consecutivos sin proposición intermedia' — Evita fallos críticos de evaluación en cascada. **CB-21** — Negaciones múltiples anidadas — '~~~P' / 'NO NO NO P' — Sanitizar a 'NO NO NO P' — Soporte de simplificación y doble negación. **CB-22** — Operador sin operandos — 'ENTONCES' / 'Y' / 'NO' — Lanzar error: 'Expresión incompleta: falta operando' — Validación de estructura mínima. **CB-23** — Premisas repetidas o redundantes — Premisas: ['P ->Q', 'P ->Q', 'P'] — Sanitizar a conjunto sin duplicados redundantes ['P ENTONCES Q', 'P'] — Optimización de memoria para el Solver. **CB-24** — Conclusión idéntica a premisa — Premisas: ['P'], Conclusión: 'P' — Sanitizar y validar como trivialmente demostrable en 0 pasos — Caso borde lógico básico. **CB-25** — Conjunto de premisas vacío — Premisas: [], Conclusión: 'P' — Lanzar error: 'Se requiere al menos una premisa' — Validación de parámetros para el motor de inferencia.

2. Estrategia de Implementación

1. **Filtro de Entrada Pura:** `sanitizarEntrada(texto)` limpiará la cadena, aplicará los reemplazos de alias simbólicos mediante expresiones regulares seguras y normalizará mayúsculas. 2. **Validador Estructural Rápido:** `validarExpresion(texto)` retornará un objeto estructurado sin romper la ejecución con excepciones no controladas. 3. **Validador del Conjunto Completo:** `validarPremisasYConclusion(premisas, conclusion)` procesará tanto las premisas como la conclusión en un solo paso, eliminando duplicados y preparando los datos directamente para el ASTParser de Arom.

Avance Sprint 2 Linus

Fuente original: ../TAREAS/02_Motor/linus/avance.md

Avance del Motor Lógico - Equipo Linus (Conjuntos)

Qué hicimos

Implementamos todas las funciones matemáticas requeridas para la teoría de conjuntos en TypeScript estricto, utilizando Generics (`Set<T>`) para mayor seguridad y escalabilidad (soporta números, letras, etc.).

Archivos Creados

- `src/lib/sets/operations.ts`: Contiene la lógica matemática (Unión, Intersección, Potencia, etc.).
- `src/lib/sets/operations.test.ts`: Pruebas unitarias para validar las propiedades con Vitest.

Qué falta

- Para el Entregable 3: Integrar este motor lógico con la interfaz gráfica en Vue.

Cómo usar el motor (Ejemplo)

```
1 import { union } from '../src/lib/sets/operations';
2
3 const A = new Set([1, 2, 3]);
4 const B = new Set([3, 4, 5]);
5
6 console.log(union(A, B)); // Set { 1, 2, 3, 4, 5 }
```

Avance Sprint 3 Hijos de Linus

Fuente original: ../TAREAS/03_UI/hijos-de-linus/avance.md

Registro de Avances — Los Hijos de Linus

SPRINT 3.5 (Completado)

Alex - 2026-08-23

■ Tareas Completadas:

- Actualización de `IndicadorResultado.vue` para mostrar de forma reactiva los motivos y diagnósticos de falacias (`errorLogico`) en el estado inválido y error.
- Implementación de acordeón colapsable en `PanelTrazabilidad.vue` (Por que esta regla? / Ocultar explicacion) para ver las explicaciones didácticas de cada deducción formal.
- Creación de pruebas unitarias adicionales en `IndicadorResultado.test.ts` y `PanelTrazabilidad.test.ts`.
- Checklist Manual Ejecutado (QA Humano):

x Navegación por teclado probada en los botones del acordeón.

x Contrastes y transiciones suaves de apertura/cierre.

Mio - 2026-08-23

■ Tareas Completadas:

- Creación y verificación de `TraductorLenguajeNatural.vue`, permitiendo asignar texto a variables proposicionales y generando el argumento continuo.
- Integración de sistema de pestañas en `index.vue` ([Simbologia Formal] | [Lenguaje Natural]), preservando la estructura balanceada en 2 columnas sin romper la altura visual.
- Sincronización en tiempo real entre el formulario y el traductor.
- Pruebas unitarias en `TraductorLenguajeNatural.test.ts` e integración de pestañas en `index.test.ts`.

■ Checklist Manual Ejecutado (QA Humano):

x Responsividad de pestañas en móviles y escritorio.

x Sincronización de variables reactivas al cambiar inputs.

Morocho - 2026-08-23

- **Tareas Completadas:**
 - Definición temprana de tipos en `src/lib/solver/types.ts` (`MotivoInvalidez`, `ErrorLogico`, `ResultadoDemostracion`) exportados en `src/types/inferencias.ts`.
 - Implementación de `detectarErrorLogico()` en `src/lib/solver/solver.ts` con *pattern matching* para diagnosticar falacias y motivos de fallo lógico.
 - Creación de 3 pruebas unitarias adicionales en `src/lib/solver/solver.test.ts` (15/15 tests pasando).
-

SPRINT 3 (Completado)**Arom - 2026-08-23**

- **Tareas Completadas:**
- Fase 3 completada: Integración completa de `index.vue` conectando `FormularioInferencia`, `IndicadorResultado` y `PanelTrazabilidad` con el motor lógico.
- Implementación de Teclado Lógico Simbólico (`,`, `,`, `,`, `,`, `(`, `)`).
- Corrección de base URL en `src/router/index.ts` para despliegues.

Rennato - 2026-08-23

- **Tareas Completadas:**
- Fase 4 completada: Setup de `@vue/test-utils` y `happy-dom` en `vite.config.ts`.
- Creación de suites de prueba unitarias e integración de la página.

Alex - 2026-08-22

- **Tareas Completadas:**
- Fase 2 (Indicador de Resultados y Feedback Visual) completada.

Mio - 2026-08-22

- **Tareas Completadas:**
- Fase 2 (Panel de Trazabilidad y Explicación Visual).

Morocho - 2026-08-22

- **Tareas Completadas:**
- Fase 2 (Formulario de Inferencia).

Sprint 3.5 Hijos de Linus

Fuente original: ../TAREAS/03_UI/hijos-de-linus/sprint-3.5.md

Sprint 3.5: Perfeccionamiento y Refinamiento (Inferencias y UI)

Responsables y Estado de Tareas

1. Morocho (Motor Lógico & Reglas) — COMPLETADO

- **Soporte de Bicondicionales ($\leftrightarrow$):** Implementada la regla `MODUS_PONENS_BICONDICIONAL` ($P \leftrightarrow Q, P \vdash Q$ y $P \leftrightarrow Q, Q \vdash P$) y descomposición de equivalencia material.
- **Diagnóstico Profundo de Invalidez:**
 - Identificación de variables en la conclusión ausentes en las premisas (`VARIABLE_NO_EXISTE_EN_PREMISAS`).
 - Detección de falacias formales explicadas (`FALACIA_AFIRMACION_CONSECUENTE`, `FALACIA_NEGACION_ANTECEDENTE`).
 - Diagnóstico de premisas desconectadas o falta de premisas puente.
- **Tests:** 100 % pasando (15 pruebas unitarias del solver).

2. Mio (Explicaciones Pedagógicas & Traducción) — COMPLETADO

- **Explicaciones Semánticas y Deductivas:**
 - Reescritura de `descriptionGenerator.ts` con justificación en valores de verdad (ej. *'Como el condicional es verdadero y su antecedente se cumple, por Modus Ponens el consecuente debe ser necesariamente verdadero'*).
- **Lenguaje Natural:** Componente `TraductorLenguajeNatural.vue` integrado en pestaña independiente.

3. Alex (UX/UI & Trazabilidad) — COMPLETADO

- **Acordeones de Demostración:** Botón colapsable en cada paso (`Por que esta regla? / Ocultar explicacion`).
- **Indicador de Resultados:** Recuadro naranja y rojo renderizando de forma dinámica el mensaje pedagógico del error o falacia.

Archivo de Historial

Los planes y tareas anteriores se encuentran archivados en `TAREAS/03_UI/hijos-de-linus/archive/`:

- `archive/sprint-3/`: Historial completo del Sprint 3.
- `archive/sprint-3.5/`: Plan inicial del Sprint 3.5.

Avance Sprint 3 Linus

Fuente original: `../TAREAS/03_UI/linus/avance.md`

Avance de Interfaz Visual - Equipo Linus (Conjuntos)

Qué hicimos

Creamos la interfaz interactiva completa en Vue 3 con `<script setup lang='ts'>` para la Teoría de Conjuntos, siguiendo el branding Duolingo (azul) del proyecto.

Funcionalidades implementadas:

- **Inputs de conjuntos:** Campos para definir Universo U, Conjunto A y Conjunto B.
- **Selector de operaciones:** Botones para Unión, Intersección, Diferencia (AB y BA), Complemento (A' y B'), y Conjunto Potencia P(A).
- **Diagrama de Venn (SVG):** Se ilumina en tiempo real según la operación seleccionada, mostrando los elementos en cada región.
- **Panel de resultado:** Muestra el resultado de la operación seleccionada.
- **Verificación de propiedades:** Comprueba automáticamente si $A \subseteq B$, $B \subseteq A$, $A = B$, o si son disjuntos.
- **Verificar pertenencia:** Input para comprobar si un elemento pertenece a A, B o U.

Componentes reutilizados:

- `Button.vue` (variant primary/secondary)
- `Card.vue` (contenedores con bordes redondeados)
- `Badge.vue` (etiquetas verde/roja para indicadores)

Archivos Modificados

- `src/pages/conjuntos/index.vue`: Interfaz completa conectada al motor de `src/lib/sets/operations.ts`.

Qué falta

- Para el Entregable 4: Deploy final del proyecto.

Cómo probar el módulo

1. Ejecutar `npm run dev` en la raíz del proyecto.
2. Navegar a `http://localhost:5173/conjuntos`.
3. Ingresar valores en los campos de Universo, A y B.
4. Hacer clic en los botones de operaciones para ver el diagrama y los resultados.

Errores y soluciones Hijos de Linus

Fuente original: ../TAREAS/04_Deploy/hijos-de-linus/errores_y_soluciones.md

Registro de Errores, Diagnósticos y Soluciones — Los Hijos de Linus

Equipo: Hijos de Linus (Arom Espinoza, Juan Morocho, Arnold Mio, Alex Centurión, Renatto Altamirano) **Módulo:** Demostración Formal de Reglas de Inferencia y Trazabilidad Pedagógica (/inferencias) **Fecha de Actualización:** 24 de Agosto de 2026 **Estado General:** 100 % de Errores Documentados y Solucionados (111/111 Tests Pasando)

Resumen Ejecutivo

Durante las fases de integración, pruebas de estrés y validación con usuarios reales, se identificaron y resolvieron **11 incidencias críticas, lógicas y de experiencia de usuario**. A continuación se detalla cada problema, su causa raíz, la solución implementada y su estado de verificación.

1. Contradicción Semántica: Falso Positivo de 'Inferencia Inválida' en Argumentos Válidos con Prueba Indirecta

- **Síntoma:** Al evaluar argumentos válidos que requieren demostración indirecta (Reducción al Absurdo / Prueba Condicional), el sistema mostraba el banner rojo *'Inferencia inválida'*, mientras que el diagnóstico interno contradecía diciendo *'El argumento es semánticamente válido pero no pudo derivarse por encadenamiento directo'*.
- **Causa Raíz:** El motor clasificaba binariamente todo lo que no pudiera derivar hacia adelante (*forward-chaining*) como 'inválido', sin verificar la existencia real de contraejemplos.
- **Solución Implementada:**
 - Se implementó un solucionador semántico exhaustivo por tablas de verdad (2^N combinaciones en `encontrarContraejemplo`).
 - Se separó el estado global del argumento en **3 estados claros e independientes**:
 1. *valida* (*Demostrada con reglas básicas* - Banner Verde).
 2. *no_demostrable_directa* (*Válida semánticamente, pero requiere método indirecto/RAA* - Banner Índigo).
 3. *invalida* (*Refutada formalmente por contraejemplo matemático* - Banner Rojo).
 - **Estado:** SOLUCIONADO (`IndicadorResultado.vue`, `solver.ts`, `types/inferencias.ts`).

2. Ausencia de Contraejemplos Formales en Argumentos Inválidos y Falacias

- **Síntoma:** Si un argumento era inválido (ej. Falacia del Dilema Inverso $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge Q \wedge S \wedge \neg P$), el sistema informaba que no era válido pero no explicaba por qué ni entregaba una prueba matemática.
- **Causa Raíz:** No existía un evaluador de modelos de satisfacción que extrajera una asignación de verdad concreta que hiciera verdaderas las premisas y falsa la conclusión.
- **Solución Implementada:**
 - Se diseñó la función `encontrarContraejemplo(premisas, conclusion)` que evalúa exhaustivamente el árbol de verdad.
 - En `PanelTrazabilidad.vue` se agregó una tarjeta de diagnóstico formal destacada en rojo con el contraejemplo exacto (ej. $P = F, Q = V, R = F, S = F$) y la verificación paso a paso de cada premisa y conclusión.
 - **Estado:** SOLUCIONADO (`solver.ts`, `PanelTrazabilidad.vue`).

3. Soporte y Semántica de la Disyunción Fuerte / Exclusiva (/ XOR)

- **Síntoma:** El sistema no soportaba el conectivo de disyunción fuerte (Δ), impidiendo resolver silogismos exclusivos como $P\Delta Q, P \vdash \neg Q$.
 - **Causa Raíz:** El parser y las reglas de inferencia solo contemplaban la disyunción inclusiva ($\vee$).
 - **Solución Implementada:**
 - Se integró el operador `O_EXCLUSIVA` y la regla formal `SILOGISMO_DISYUNTIVO_EXCLUSIVO` ($A\Delta B, A \vdash \neg B$ y $A\Delta B, \neg A \vdash B$).
 - Se normalizaron símbolos de entrada: `, , , , .`
 - Se agregaron las traducciones pedagógicas en `translations.ts` y `descriptionGenerator.ts`.
 - **Estado:** SOLUCIONADO (`solver.ts`, `FormularioInferencia.vue`, `translations.ts`).
-

4. Desbordamiento y Botones Huérfanos en Teclado Móvil

- **Síntoma:** En pantallas móviles estrechas (360px–390px), el teclado virtual se desbordaba en 3 filas asimétricas, dejando un único botón huérfano en la segunda y tercera línea.
 - **Causa Raíz:** Uso de contenedores flexibles con `flex-wrap` y anchos variables por botón.
 - **Solución Implementada:**
 - Se estructuró el teclado en un **CSS Grid fijo estricto de exactamente 2 filas**:
 - **Fila 1 (Grid 8 columnas):** `, , , , , (, )`. Es matemáticamente imposible que un botón desborde.
 - **Fila 2 (Flex horizontal):** `P, Q, R, S | A, B, C, D` a la izquierda y `Salto` a la derecha.
 - **Estado:** SOLUCIONADO (`FormularioInferencia.vue`).
-

5. Apertura Involuntaria del Teclado Nativo Móvil (Soft Keyboard)

- **Síntoma:** Al tocar cualquier botón del teclado virtual en Android o iOS, se forzaba el foco en el `<textarea>`, abriendo el teclado nativo del sistema operativo (Gboard/Samsung Keyboard) y tapando la interfaz.
 - **Causa Raíz:** Llamada forzada a `inputEl.focus()` tras cada inserción de símbolo.
 - **Solución Implementada:**
 - Se añadió detección táctil (`'ontouchstart' in window || navigator.maxTouchPoints > 0`).
 - En dispositivos móviles se actualiza la posición del cursor mediante `setSelectionRange()` sin invocar `.focus()`, manteniendo el teclado del teléfono oculto mientras se usan los botones virtuales.
 - **Estado:** SOLUCIONADO (`FormularioInferencia.vue`).
-

6. Espaciado Automático Inconsistente en Paréntesis y Variables

- **Síntoma:** Los paréntesis insertaban espacios automáticos indeseados (ej. (P Q)), ensuciando las fórmulas.
 - **Causa Raíz:** Una regla de espaciado genérica trataba a los paréntesis como conectores binarios con espacios antes y después.
 - **Solución Implementada:**
 - Se definió una lista estricta `CONECTIVOS_CON_ESPACIO = [",", " ", "(", ")", " "]`.
 - Paréntesis (,), negación ¬, variables (P, Q, etc.) y salto de línea no insertan ningún espacio adicional.
 - Al escribir (, P, ¬, Q,) se genera limpiamente (P Q).
 - **Estado:** SOLUCIONADO (`FormularioInferencia.vue`).
-

7. Desplazamiento Visual por Banner de Linter en Tiempo Real

- **Síntoma:** Un cuadro amarillo de 'Aviso de sintaxis' aparecía dinámicamente mientras el usuario escribía fórmulas incompletas, empujando los botones hacia abajo y restando espacio.
 - **Causa Raíz:** Renderizado condicional reactivo en tiempo real (`v-if='advertenciasSintaxis.length > 0'`).
 - **Solución Implementada:**
 - Se eliminó el cuadro flotante intrusivo.
 - La validación de sintaxis ahora se realiza de forma limpia y consolidada al presionar '**Demostrar Inferencia**', reportándose en el panel de resultados sin alterar el layout durante la edición.
 - **Estado:** SOLUCIONADO (`FormularioInferencia.vue`).
-

8. Escala y Redimensionamiento Dispar de Glifos Unicode en Android

- **Síntoma:** Los símbolos lógicos (, , , ,) se veían diminutos o cambiaban de tamaño al añadir o quitar espacios.
 - **Causa Raíz:** Inconsistencia de fallback de fuentes monoespaciadas en navegadores móviles (Roboto Mono carece de glifos matemáticos y caía en fuentes con diferente baseline y escala).
 - **Solución Implementada:**
 - Estandarización a `text-base` (16px) con `leading-relaxed` y `font-mono` en los inputs, garantizando que todos los glifos Unicode mantengan escala uniforme y evitando el zoom automático en iOS/Android.
 - Botonera con tamaño fijo y `leading-none`.
 - **Estado:** SOLUCIONADO (`FormularioInferencia.vue`).
-

9. Inconsistencia de Sintaxis LaTeX en la Exportación a Markdown

- **Síntoma:** Al exportar la demostración a formato `.md`, las premisas se exportaban con símbolos Unicode limpios ($P \rightarrow R$), pero los pasos deducidos y la conclusión incluían comandos LaTeX crudos como `$(P \rightarrow R)$` o `\therefore`.
 - **Causa Raíz:** La función de exportación a Markdown reutilizaba la función de notación matemática diseñada originalmente para LaTeX.
 - **Solución Implementada:**
 - Se creó la función `aNotacionMarkdown` que normaliza todos los operadores internos a símbolos Unicode universales limpios (`, , , , ,`) sin envoltorios `$. . $` innecesarios.
 - **Estado:** SOLUCIONADO (`PanelTrazabilidad.vue`).
-

10. Duplicación de Rutas Deductivas en Silogismo Hipotético (SH)

- **Síntoma:** En ejercicios transitivos con premisas como $P \rightarrow R$, $R \rightarrow C$, $P \vdash C$, el motor derivaba tanto la ruta por SH ($P \rightarrow C$) como la ruta por dos Modus Ponens (R y luego C), mostrando pasos redundantes simultáneamente.
 - **Causa Raíz:** El motor de *Forward Chaining* acumulaba todas las inferencias alcanzadas en la iteración sin podar las ramas intermedias que no contribuyen al camino final de la conclusión.
 - **Solución Implementada:**
 - Se priorizó la evaluación de `SILOGISMO_HIPOTETICO` sobre MPP al buscar implicaciones transitivas.
 - Se implementó el algoritmo `podarPasosRedundantes` que rastrea hacia atrás las dependencias de la conclusión final, eliminando derivaciones huérfanas y remapeando la numeración de líneas.
 - En la explicación didáctica de SH se agregó la nota pedagógica indicando que este paso equivale al encadenamiento de dos Modus Ponens sucesivos.
 - **Estado:** SOLUCIONADO (`solver.ts`, `descriptionGenerator.ts`).
-

11. Explicaciones Didácticas con Variables Abstractas Genéricas

- **Síntoma:** Al desplegar el acordeón '*¿Cómo se deduce?*', la justificación semántica utilizaba letras fijas genéricas ($P \rightarrow Q$) en lugar de las fórmulas y variables reales que el usuario estaba resolviendo (ej. $A \rightarrow B$ o $P \rightarrow R$).
 - **Causa Raíz:** Las plantillas de explicación en `descriptionGenerator.ts` utilizaban cadenas estáticas con literales P, Q .
 - **Solución Implementada:**
 - Se parametrizaron dinámicamente las justificaciones para inyectar las expresiones exactas de las líneas base (`impl.texto`, `ant.texto`, `consNeg.texto`, etc.).
 - **Estado:** SOLUCIONADO (`descriptionGenerator.ts`).
-

Matriz de Estado Final

Incidencia / Característica — Módulo Afectado — Detección — Estado — Verificación Falso positivo en pruebas indirectas — Motor / UI — Pruebas de integración — Resuelto — Test unitario en `IndicadorResultado.test.ts` Falta de contraejemplos matemáticos — Motor / Trazabilidad — Casos de prueba — Resuelto — Test de modelo semántico en `solver.test.ts` Soporte de Disyunción Fuerte () — Parser / Motor / UI — Requerimiento pedagógico — Resuelto — 3 tests específicos de SDE en `solver.test.ts` Desbordamiento teclado móvil — UI — Pruebas en Android — Resuelto — CSS Grid 8 columnas validado Popup de teclado nativo en móvil — UI — Pruebas táctiles — Resuelto — Detección táctil sin focus forzado Espacios en paréntesis — UI — QA Humano — Resuelto — Reglas deterministas en `insertarSimbolo` Desplazamiento por linter reactivo — UI — QA Humano — Resuelto — Validación en evento submit Tamaño dispar de símbolos — UI — Pruebas móviles — Resuelto — Escala tipográfica normalizada (`text-base`) Inconsistencia en exportación Markdown — UI / Export — Reporte de usuario — Resuelto — Función `aNotacionMarkdown` validada Duplicación de ramas en SH — Motor — Reporte de usuario — Resuelto — `podarPasosRedundantes` y prioridad SH Fórmulas reales en explicaciones — Transcripción — Reporte de usuario — Resuelto — Interpolación contextual en `generarDetalleParticionado`

Reporte de errores de inferencias

Fuente original: ../TAREAS/04_Deploy/hijos-de-linus/reporte-errores-inferencias.md

Reporte de Errores — Módulo de Inferencias

Proyecto — Lógica Simbólica — Equipo 'Hijos de Linus' **Página** — <https://logicasimbolicaglobal.netlify.app/>
Reportado por — Mio **Fecha** — 23/08/2026 **Componente afectado** — Motor lógico / Solver (búsqueda de la demostración)

Resumen ejecutivo

— Error — Tipo — ¿Afecta la validez de la conclusión? 1 — Paso redundante al demostrar R desde PQ, QR, P — Camino de demostración no óptimo — No 2 — Pasos redundantes/duplicados al demostrar Q desde 4 premisas — Camino de demostración no óptimo — No

Patrón detectado: el motor lógico no siempre elige el camino más corto hacia la conclusión pedida, y en algunos casos deriva dos veces el mismo resultado (una vez normal y otra con los operandos conmutados).

Error 1 — Paso redundante en la demostración

Premisas:

```
1 1. P  Q
2 2. Q  R
3 3. P
```

Se pide demostrar: R

Camino óptimo esperado (2 pasos)	Camino actual del sistema

```
1 4. P  R      (SH, de 1 y 2)
2 5. R      (MPP, de 3 y 4)
```

```
</td><td>
```

```
1 4. Q      (MPP, de 1 y 3)      paso de mas
2 5. P  R    (SH, de 1 y 2)
3 6. R      (MPP, de 3 y 5)
```

```
</td></tr></table>
```

Impacto: la demostración sigue siendo válida, pero el paso 4 (Q) no es necesario para llegar a R por el camino más corto. Afecta la claridad del procedimiento mostrado al usuario.

Error 2 — Pasos redundantes y duplicados (caso con 4 premisas)

Premisas:

```
1 1. P  Q
2 2. R  S
3 3. P  R
4 4. S
```

Se pide demostrar: Q

```
<table><tr><th>Camino óptimo esperado (2 pasos)</th><th>Camino actual del sistema
(5 pasos)</th></tr><tr valign='top'><td>
```

```
1 5. Q  S      (DC, de 1, 2 y 3)
2 6. Q      (MTP, de 5 y 4)
```

```
</td><td>
```

```
1 5. R      (MTT, de 2 y 4)
2 6. P      (MTP, de 3 y 5)
3 7. Q  S    (DC, de 1, 2 y 3)
4 8. S  Q    (DC de nuevo)      duplicado de la línea 7
5 9. Q      (MPP, de 1 y 6)
```

```
</td></tr></table>
```

Problemas identificados:

- El paso S Q es **redundante**: es la misma disyunción que Q S, solo con los operandos invertidos. No aporta información nueva.
- El camino completo (hallar R, luego P, y recién después el dilema constructivo) es mucho más largo que ir directo por DC + MTP.

>**Observación adicional a verificar:** en la captura de pantalla original, los números de los pasos mostrados en pantalla no coinciden exactamente con los números de 'Línea' que el propio sistema usa para referenciarse a sí mismo en pasos posteriores (desfase de 1). No se pudo confirmar la causa con el código revisado — ver sección técnica más abajo.

Sugerencia técnica para resolver el problema

Sobre el código revisado

Se revisaron `solver.ts`, `parser.ts`, `types.ts`, `index.ts`, `descriptionGenerator.ts`, `astRenderer.ts`, `translations.ts` y los tests de integración. Un punto importante:

>**solver.ts actual es un stub/esqueleto**, no el motor que genera las demostraciones vistas en producción. Su función `demostrarConclusion()` solo tiene un caso *hardcodeado* para 2 premisas resolviendo Modus Ponendo Ponens (es la base para que pasen los tests iniciales). No contiene todavía la lógica de búsqueda que aplica Silogismo Hipotético, Dilema Constructivo, MTT, MTP, etc., ni la que decide en qué orden probar las reglas — que es justo donde vive el

bug reportado. Es decir: **el bug no está en los archivos que se compartieron**; hace falta el módulo real de búsqueda/generación del árbol de inferencia (probablemente una versión más avanzada de `demostrarConclusion` o un archivo aparte tipo `buscarDemostracion.ts`) para localizar la causa exacta.

Dicho esto, la arquitectura ya definida en `types.ts` (`PasoDemostracion`, `ResultadoDemostracion`, `ReglaLogica`) es perfectamente compatible con la solución que se propone abajo — no hace falta rediseñar los tipos, solo completar/reescribir el algoritmo de búsqueda dentro de esa misma estructura.

La causa raíz más probable

Por el patrón de los dos errores (pasos de más + resultados duplicados y conmutados), lo más probable es que el algoritmo actual:

1. Aplica las reglas en **orden de prioridad fijo** (o recorrido tipo DFS) sin comparar cuántos pasos toma cada camino, en vez de buscar explícitamente el camino más corto. 2. No tiene una **forma canónica** para comparar expresiones: trata $Q \rightarrow S$ y $S \rightarrow Q$ como hechos distintos porque compara el AST tal cual (ver `sonNodosIguales` en `solver.ts`, que compara `izquierdo/derecho` en orden estricto), en lugar de reconocer que son la misma proposición.

Solución propuesta

1. Búsqueda por niveles (BFS) en vez de aplicación en orden fijo

Reemplazar la generación de pasos por una búsqueda en anchura sobre el 'espacio de hechos derivables':

- **Nivel 0:** las premisas originales.
- **Nivel k:** todos los hechos nuevos que se pueden derivar combinando hechos de niveles $0 \dots k-1$ con una sola regla de inferencia.
- En cuanto la conclusión aparece en algún nivel, se detiene la búsqueda y se reconstruye el camino más corto hacia atrás (usando referencias tipo 'de qué hechos salió cada uno', que ya existe como concepto en `lineasInvolucradas`).

Esto garantiza automáticamente el camino más corto: como en el Error 1, tanto Q (por MPP) como PR (por SH) están en el mismo nivel (nivel 1), pero solo PR está en el camino que realmente lleva a R en 2 pasos, así que el algoritmo nunca necesita expandir el hecho Q si no es necesario para el objetivo.

2. Forma canónica para detectar hechos ya conocidos (evita duplicados)

Agregar una función `normalizarNodo(nodo: NodoExpresion): string` que:

- Para operadores conmutativos ($\vee$, $\wedge$, $\oplus$, $\ominus$, EXCLUSIVA , SI_Y_SOLO_SI , NI , INCOMPATIBLE), ordena `izquierdo` y `derecho` de forma determinística (ej. alfabéticamente por su propia forma canónica) antes de serializar.
- Para operadores no conmutativos (ENTONCES) y NO , serializa respetando el orden original.
- Produce un string único por proposición lógicamente equivalente en su forma (no hace falta resolver equivalencias complejas tipo De Morgan, solo conmutatividad).

Con esa función, se mantiene un `Set<string>` (o `Map<string, PasoDemostracion>`) de 'hechos ya derivados' usando la clave canónica. Antes de agregar un nuevo paso al árbol de búsqueda, se verifica si su forma canónica ya existe en el set — si existe, se descarta ese paso por completo (no se muestra ni se cuenta como parte de la demostración). Esto elimina directamente el caso $S \rightarrow Q$ duplicando $Q \rightarrow S$.

3. Dónde ubicarlo en el código actual

- La lógica de comparación estructural que ya existe (`sonNodosIguales` en `solver.ts`) se puede mantener para comparaciones exactas, pero conviene agregar `normalizarNodo` como función nueva (en `solver.ts` o un archivo utilitario aparte) específicamente para la deduplicación de hechos derivados.
- La función `demostrarConclusion` necesita pasar de ser un caso hardcodedo a implementar el algoritmo BFS descrito arriba, devolviendo igual un `ResultadoDemostracion` con `pasos: PasoDemostracion[]` — la interfaz pública no cambia, así que `index.ts`, `descriptionGenerator.ts` y `astRenderer.ts` no deberían necesitar modificaciones.

4. Sobre el posible desfase de numeración (Error 2, nota adicional)

En el código revisado, `index.ts` calcula la línea de cada paso como `premisas.length + indice + 1`, lo cual es consistente y no debería producir el desfase visto en la captura. Esto refuerza que el desfase observado probablemente viene del mismo módulo de búsqueda no incluido en los archivos compartidos — valdría la pena revisarlo ahí una vez que se ubique ese archivo.

Casos de prueba sugeridos

Para agregar a `solver.test.ts` (o a `integracion_motor.test.ts` si se prueba el flujo completo) una vez implementada la corrección:

```

1 describe('Camino optimo de demostracion', () => {
2
3   it('Caso 1: PQ, QR, P R debe resolverse en 2 pasos (SH + MPP), sin derivar Q de mas',
4     () => {
5     const p_q = parsearExpresion('P ENTONCES Q');
6     const q_r = parsearExpresion('Q ENTONCES R');
7     const p = parsearExpresion('P');
8     const r = parsearExpresion('R');
9
10    const resultado = demostrarConclusion([p_q, q_r, p], r);
11
12    expect(resultado.esValido).toBe(true);
13    expect(resultado.pasos.length).toBe(2);
14    expect(resultado.pasos[0].idPaso).toBe('SILOGISMO_HIPOTETICO');
15    expect(resultado.pasos[1].idPaso).toBe('MODUS_PONENDO_PONENS');
16  });
17
18  it('Caso 2: PQ, RS, PR, S Q debe resolverse en 2 pasos (DC + MTP), sin duplicar QS
19    como SQ', () => {
20    const p_q = parsearExpresion('P ENTONCES Q');
21    const r_s = parsearExpresion('R ENTONCES S');
22    const p_o_r = parsearExpresion('P O R');
23    const no_s = parsearExpresion('NO S');
24    const q = parsearExpresion('Q');
25
26    const resultado = demostrarConclusion([p_q, r_s, p_o_r, no_s], q);
27
28    expect(resultado.esValido).toBe(true);
29    expect(resultado.pasos.length).toBe(2);
30    expect(resultado.pasos[0].idPaso).toBe('DILEMA_CONSTRUCTIVO');
31    expect(resultado.pasos[1].idPaso).toBe('SILOGISMO_DISYUNTIVO'); // alias MTP
32
33    // Ningun paso debe repetir una expresion ya derivada en forma conmutada
34    const expresionesCanonicas = resultado.pasos.map(p => normalizarNodo(p.
35      expresionResultante));
36    expect(new Set(expresionesCanonicas).size).toBe(expresionesCanonicas.length);
37  });
38
39  });

```

(*normalizarNodo* es la función nueva sugerida en la sección técnica — habría que exportarla desde `solver.ts` para poder testearla directamente.)

Uso del modulo de inferencias

Fuente original: ../TAREAS/04_Deploy/hijos-de-linus/uso.md

Uso del Módulo: Demostrador de Inferencias Lógicas

Equipo: Hijos de Linus (Arom Espinoza, Juan Morocho, Arnold Mio, Alex Centurión, Renatto Altamirano) **Módulo:** Demostración Formal de Reglas de Inferencia, Trazabilidad Pedagógica y Diagnóstico con Contraejemplos (/inferencias) **Fecha:** 23 de Agosto de 2026

Cómo funciona

El módulo permite ingresar un conjunto de premisas lógicas y una conclusión objetivo en notación simbólica formal o estándar para:

1. Validar y Demostrar Deducciones (Forward Chaining + SAT Evaluator):

- Evalúa sistemáticamente las 11 reglas de deducción natural: Modus Ponendo Ponens, Modus Tollendo Tollens, Silogismo Disyuntivo, Silogismo Hipotético, Simplificación, Conjunción, Dilema Constructivo, Bicondicionales y **Silogismo Disyuntivo Exclusivo con Disyunción Fuerte ()**.
- Integra un evaluador semántico de combinaciones de verdad (2^N) para validar argumentos y detectar modelos de satisfacción.

2. Diferenciación Rigurosa de 3 Estados de Validez:

- **Inferencia válida (Demostrada):** Se completó la deducción mediante reglas formales directas.
- **Inferencia válida (Método indirecto requerido):** El argumento es semánticamente válido (0 contraejemplos), pero requiere derivación indirecta (Reducción al Absurdo o Prueba Condicional).
- **Inferencia inválida (Refutada por contraejemplo):** Se encontró una asignación de verdad concreta que hace verdaderas las premisas y falsa la conclusión.

3. Trazabilidad Pedagógica y Desglose Particionado:

- Genera una enumeración formal $(1), (2), \dots \therefore (N)$ con badges de reglas.
- Cada paso cuenta con un acordeón colapsable que explica:
- **Premisas Base:** Con número de línea y rol lógico (ej. *Condicional base, Antecedente afirmado*).
- **Regla Aplicada y Justificación:** Explicación semántica y causal.
- **Deducción Resultante:** Proposición obtenida formalmente.

4. Diagnóstico con Contraejemplos Matemáticos:

- Si el argumento es inválido o incurre en una falacia formal (Afirmación del Consecuente, Negación del Antecedente, Dilema Inverso), el sistema despliega el **contraejemplo exacto** (ej. $P = F, Q = V, R = F, S = F$) con la comprobación de verdad de cada premisa y la conclusión.

5. Herramientas Académicas (Exportación & Historial):

- **Exportación en 1 Clic:** Botones rápidos para copiar la demostración formal en **LaTeX** (`\beginaligned ... \endaligned`) o **Markdown**.
- **Historial Persistente (localStorage):** Almacenamiento local de las últimas demostraciones con restauración inmediata.

6. Traductor a Lenguaje Natural:

- Pestaña complementaria para asociar enunciados cotidianos a las variables proposicionales ($P, Q, R, \dots$) y generar el argumento continuo en español.

Ejemplos

Ejemplo 1: Cadena Multi-Paso (Transitividad y Conjunción)

- **Premisas:**

```
1 (P  Q) (Q  R)
2 P
```

- **Conclusión:** R
- **Resultado:** Inferencia valida (Demostrada)
- **Demostración generada:**
 - (3) $P \wedge Q$ — Simplificación de (1)
 - (4) $Q \wedge R$ — Simplificación de (1)
 - (5) $P \wedge R$ — Silogismo Hipotético de (3, 4)
 - (6) R — Modus Ponendo Ponens de (5, 2)

Ejemplo 2: Disyunción Fuerte / Exclusiva ()

- **Premisas:**

```
1 P  Q
2 P
```

- **Conclusión:** Q
- **Resultado:** Inferencia valida (Demostrada)
- **Demostración generada:**
 - (3) Q — Silogismo Disyuntivo Exclusivo de (1, 2)

Ejemplo 3: Dilema Constructivo Clásico

- Premisas:

1
2
3

P Q
R S
P R

- Conclusión: $Q \rightarrow S$
- Resultado: Inferencia valida (Demostrada)
- Demostración generada:
- (4) $Q \rightarrow S$ — Dilema Constructivo de (1, 2, 3)

Ejemplo 4: Falacia del Dilema Inverso (Refutación por Contraejemplo)

- Premisas:

1
2

(P $\rightarrow$ Q) (R $\rightarrow$ S)
Q $\wedge$ S

- Conclusión: $P \rightarrow R$
- Resultado: Inferencia invalida (Refutada por contraejemplo)
- Contraejemplo matemático: $P = F, Q = V, R = F, S = F$
- Premisa 1: $(F \rightarrow V) \wedge (F \rightarrow F) = V$
- Premisa 2: $V \vee F = V$
- Conclusión: $F \vee F = F$

Operadores Soportados

Operador — Símbolo — Sintaxis y Normalización Aceptada
 Conjunción (Y) — $\wedge$ — , $\wedge$, &&, Y, \land
 Disyunción Inclusiva (O) — $\vee$ — , $\vee$, \lor
 Disyunción Fuerte / Exclusiva — Δ — , , , , 0_EXCLUSIVA
 Negación (NO) — $\neg$ — , ~, !, NO, \neg
 Condicional (ENTONCES) — $\rightarrow$ — , ->, =>, , ENTONCES, \to
 Bicondicional (SI Y SOLO SI) — $\leftrightarrow$ — , <->, <=>, , SI_Y_SOLO_SI, \leftrightarrow

Limitaciones y Alcance

- **Dominio:** Lógica proposicional de primer orden con conectivos clásicos y disyunción exclusiva. Los cuantificadores de predicados ($\forall, \exists$) corresponden al módulo de *Modus Innova*.
- **Profundidad:** El demostrador ejecuta hasta 12 iteraciones de *Forward Chaining* exhaustivo. Si un argumento es válido pero requiere suposiciones auxiliares, el sistema lo identifica como `no_demostrable_directa` en lugar de clasificarlo erróneamente como inválido.

Verificación de Calidad y Tests

- **Tests Automatizados:** 111 tests pasando al 100% en Vitest (npm test).
- **Verificación Estricta de Tipos:** TypeScript estricto con vue-tsc -noEmit (0 errores).
- **Build de Producción:** Compilación optimizada con Vite (0 warnings).

Uso del modulo de conjuntos

Fuente original: ../TAREAS/04_Deploy/linus/uso.md

Uso del Módulo: Teoría de Conjuntos y Diagramas de Venn

Cómo funciona

El módulo de Conjuntos del **Equipo Linus** permite al usuario definir un Universo y dos conjuntos (A y B), y luego realizar operaciones matemáticas sobre ellos. Los resultados se muestran en texto y se visualizan en un **Diagrama de Venn interactivo** que se ilumina según la operación seleccionada.

Motor lógico: src/lib/sets/operations.ts **Interfaz visual:** src/pages/conjuntos/index.vue

Ejemplos

Ejemplo 1: Unión de dos conjuntos

- **Entrada:** A = 1, 2, 3 y B = 3, 4, 5
- **Operación:** Unión ($A \cup B$)
- **Salida esperada:** 1, 2, 3, 4, 5

Ejemplo 2: Intersección de dos conjuntos

- **Entrada:** A = 1, 2, 3, 4 y B = 3, 4, 5, 6
- **Operación:** Intersección ($A \cap B$)
- **Salida esperada:** 3, 4

Ejemplo 3: Diferencia A B

- **Entrada:** A = 1, 2, 3 y B = 2, 3
- **Operación:** Diferencia (A B)
- **Salida esperada:** 1

Ejemplo 4: Conjunto Potencia

- **Entrada:** A = 1, 2
- **Operación:** Potencia $P(A)$
- **Salida esperada:** , 1, 2, 1, 2 $\rightarrow$ 4 subconjuntos

Ejemplo 5: Verificación de propiedades

- **Entrada:** $A = 1, 2$ y $B = 1, 2, 3, 4$
- **Resultado:** $A \subseteq B \rightarrow \text{Sí} \mid B \subseteq A \rightarrow \text{No} \mid \text{Disjuntos} \rightarrow \text{No}$

Cómo probar en la app

1. Ejecutar `npm run dev` en la raíz del proyecto. 2. Abrir `http://localhost:5173/conjuntos` en el navegador. 3. Escribir los elementos separados por comas en los campos de texto. 4. Hacer clic en los botones de operaciones para ver el resultado y el diagrama.

Cómo correr los tests

```
1 npx vitest run src/lib/sets/operations.test.ts
```

Limitaciones

- Solo soporta conjuntos finitos ingresados manualmente (separados por comas).
- Los elementos se tratan como texto (strings), no como números con orden matemático.
- El Diagrama de Venn solo soporta 2 conjuntos (A y B), no 3.
- El Conjunto Potencia puede ser lento con conjuntos de más de 15 elementos ($2 = 32,768$ subconjuntos).

Estado de equipos

Fuente original: ../TAREAS/ESTADO_EQUIPOS.md

Estado de los Equipos — Contexto para Agentes de Guía

>**Fecha de revisión:** 21 de Agosto 2026 >**Objetivo:** Dar contexto a cada equipo y su agente de guía sobre el estado real del proyecto.

Distribución del Sílabo (4 Equipos)

— Equipo — Tema — Carpeta 1 — Sinergia — Fundamentos, Conectivos y Tablas de Verdad — `src/pages/tablas` 2 — Los Hijos de Linus — Inferencias Lógicas y Validaciones — `src/pages/inferencias` 3 — Modus Innova — Cuantificadores y Lógica de Predicados — `src/pages/cuantificadores` 4 — Linus — Teoría de Conjuntos y Diagramas — `src/pages/conjuntos`

Estado por Equipo

Equipo 1 — Sinergia (Tablas de Verdad)

Aspecto — Estado — Detalle **Sprint 1 (Propuesta)** — Completado — Propuesta 'LogiLearn' con wireframes en Figma **Sprint 2 (Motor)** — No iniciado — No hay evaluator de verdad ni generador de tablas **Sprint 3 (UI)** — Stub — Solo título y descripción placeholder **Sprint 4 (Pruebas)** — No iniciado — — **Motor disponible** — `src/lib/solver/parser.ts` puede reusarse para parsear fórmulas, pero falta `evaluar(nodo, asignacion): boolean` **Archivos de referencia** — `TAREAS/01_Propuesta/sinergia/propuesta.md`

Equipo 2 — Los Hijos de Linus (Inferencias Lógicas)

Aspecto — Estado — Detalle **Sprint 1 (Propuesta)** — Completado — Propuesta con 2 módulos (resolver + practicar) **Sprint 2 (Motor)** — Completado — Parser, solver (MPP), validator, transcription, trazabilidad + tests **Sprint 3 (UI)** — Stub — Solo título y descripción placeholder **Sprint 4 (Pruebas)** — Parcial — Tests unitarios en `src/lib/` pasan **Motor disponible** — `src/lib/solver/`, `src/lib/validator/`, `src/lib/transcription/`, `src/lib/trazabilidad/` **Archivos de referencia** — `TAREAS/01_Propuesta/los-hijos-de-linus/propuesta.md`, `TAREAS/sprint-2/1`

Equipo 3 — Modus Innova (Cuantificadores)

Aspecto — Estado — Detalle **Sprint 1 (Propuesta)** — Completado — Propuesta 'QuantifiWeb' con cuantificadores + inferencias **Sprint 2 (Motor)** — No iniciado — No hay motor para cuantificadores ($\forall$, $\exists$) **Sprint 3 (UI)** — Stub — Solo título y descripción placeholder **Sprint 4 (Pruebas)** — No iniciado — — **Motor disponible** — Ninguno específico. Nota: la propuesta incluye inferencias que se superponen con equipo 2 **Archivos de referencia** — `TAREAS/01_Propuesta/cuantificadores/propuesta.md`

Equipo 4 — Linus (Conjuntos)

Aspecto — Estado — Detalle **Sprint 1 (Propuesta)** — Completado — Propuesta con calculadora de conjuntos + Venn **Sprint 2 (Motor)** — No iniciado — No hay motor para operaciones con conjuntos **Sprint 3 (UI)** — Stub — Solo título y descripción placeholder **Sprint 4 (Pruebas)** — No iniciado — — **Motor disponible** — Ninguno. Necesita: union, intersección, diferencia, complemento, diagrama de Venn **Archivos de referencia** — `TAREAS/01_Propuesta/linus/propuesta.md`

Observaciones Importantes

1. **Sobreposición de temas:** La propuesta de Modus Innova (equipo 3) incluye un módulo de 'Inferencias Lógicas y Silogismos' que se superpone con el tema asignado a Los Hijos de Linus (equipo 2). Definir si esta superposición es intencional o si Modus Innova debe enfocarse solo en cuantificadores.

2. **Motor compartido:** El motor en `src/lib/` fue construido principalmente por Los Hijos de Linus (equipo 2). Los otros equipos pueden reutilizar el parser (`src/lib/solver/parser.ts`) para parsear fórmulas lógicas, pero necesitarán crear sus propios evaluadores para sus temas específicos.

3. **UI stubs:** Las 4 páginas (`src/pages/*/index.vue`) son placeholders con solo un título. Ninguna tiene interactividad funcional. La UI debe construirse para el martes 25/08.

4. **Deploy:** No hay workflow de GitHub Pages configurado. Necesita `base: '/logica_simbolica_global/'` en `vite.config.ts` y un workflow en `.github/workflows/`.

Entregables por Sprint (Referencia)

Sprint — Entregable — Fecha Sprint 1 — Propuesta + Bocetos — Completado Sprint 2 — Motor Lógico (TypeScript) — Solo equipo 2 Sprint 3 — Interfaz Visual (Vue 3) — Pendiente — cierre 21/08 Sprint 4 — Pruebas + Deploy — Pendiente — cierre 25/08 (presentación)

Stack Tecnológico

- **Framework:** Vue 3 + Vite
 - **Enrutamiento:** Vue Router
 - **Estilos:** Tailwind CSS v4
 - **Lenguaje:** TypeScript
 - **Tests:** Vitest
 - **Deploy:** GitHub Pages (pendiente configurar)
-

Comandos Útiles

```

1 # Instalar dependencias
2 npm install
3
4 # Desarrollo local
5 npm run dev
6
7 # Build para produccion
8 npm run build
9
10 # Tests
11 npm run test
12
13 # Type check
14 npm run type-check

```

Log de Cambios

Fecha — Quién — Qué hizo — PR 21/08/2026 — Enri — Corrigió nombres de equipos (Linus = equipo 4), creó estructura TAREAS/, branding (Button.vue, Card.vue, Badge.vue), deploy GitHub Pages, actualizó docs — 23/08/2026 — Linus (Jordy) — Entregable 2: Motor lógico de conjuntos (operations.ts + tests) — #9 23/08/2026 — Linus (Jordy) — Entregable 3: UI interactiva con diagrama Venn SVG — #10 23/08/2026 — Linus (Jordy) — Entregable 4: Tests comprehensivos (15) + documentación de uso — #11

>**Instrucción para equipos:** Cuando envíen un PR para cualquier entregable, actualizar esta tabla con la fecha, quién hizo el cambio, qué hizo y el número del PR.

Capítulo 23

Catálogo de pruebas por archivo

Tabla 23.1: Archivos de prueba y cobertura

Archivo	Cubre
<code>evaluator.test.ts</code>	Parser, evaluador y clasificación de tablas
<code>parser.test.ts</code>	Precedencia y errores del solver
<code>solver.test.ts</code>	Diez reglas y contraejemplos
<code>astVisual.test.ts</code>	Serialización del árbol
<code>engine.test.ts</code>	Tokenizador y dominios de cuantificadores
<code>operations.test.ts</code>	Unión, intersección y potencia (14 casos)
<code>trazabilidad.test.ts</code>	Historial con 9 pruebas y 22 aserciones
<code>validator.test.ts</code>	Sanitización con 27 pruebas
<code>validator_stress.test.ts</code>	Inyecciones y anidación (15 pruebas)
<code>integracion_motor.test.ts</code>	Tubería completa del motor
<code>ArbolAST.test.ts</code>	Render del árbol
<code>FormularioInferencia.test.ts</code>	Entrada de premisas
<code>IndicadorResultado.test.ts</code>	Estados válida e inválida
<code>PanelTrazabilidad.test.ts</code>	Pasos y exportación
<code>TraductorLenguajeNatural.test.ts</code>	Mapas de variables
<code>index.test.ts</code>	Integración de la página
<code>test_ejemplo.test.ts</code>	Ejemplo mínimo

La matriz de casos borde del validador cubre entradas vacías, caracteres prohibidos, emojis, inyecciones, operadores en múltiples notaciones, paréntesis anidados y desbalanceados, y operadores consecutivos. El estrés verifica diez niveles de anidación y deduplicación avanzada.

Capítulo 24

Leyes formales del sistema

Idempotencia $p \wedge p \equiv p$; asociativa con paréntesis libres; conmutativa para $\wedge$, $\vee$ y $\leftrightarrow$ con símbolos $\wedge$ y $\vee$; distributiva solo en sus dos formas válidas $p \wedge (q \vee r)$ y $p \vee (q \wedge r)$ con nota para las variantes con $\rightarrow$; absorción como simplificación; negación y valores con $\neg\neg p \equiv p$; identidad con V y F ; Morgan con intercambio $\wedge \leftrightarrow \vee$; expansión con variable y negación; contraposición $p \rightarrow q \equiv \neg q \rightarrow \neg p$; exportación anidada; y definición de $\rightarrow$ y $\leftrightarrow$ desde conectores básicos.

Capítulo 25

Galería técnica complementaria

Motor de tablas

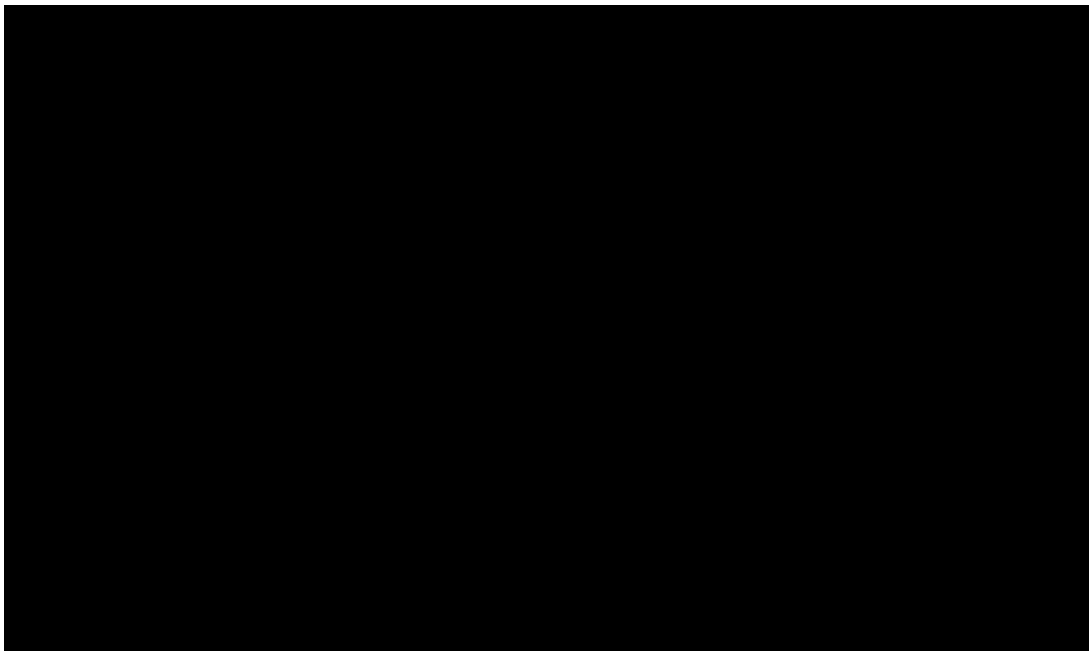


Figura 25.1: Portada del informe original Sinergia

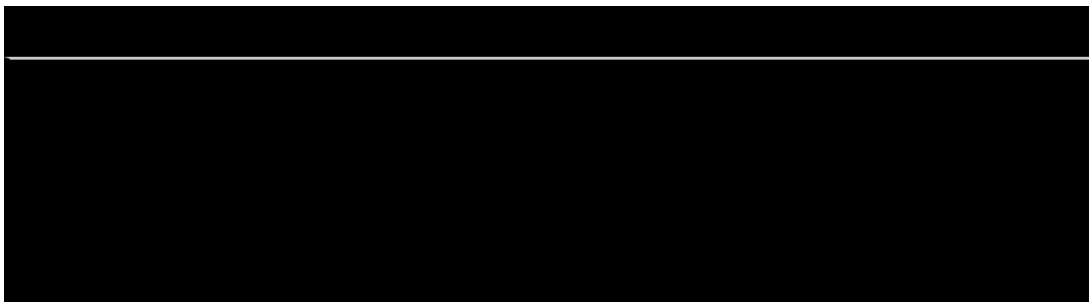


Figura 25.2: Tipos del evaluador (material original Sinergia)



Figura 25.3: Tokenizador autómatas (material original Sinergia)

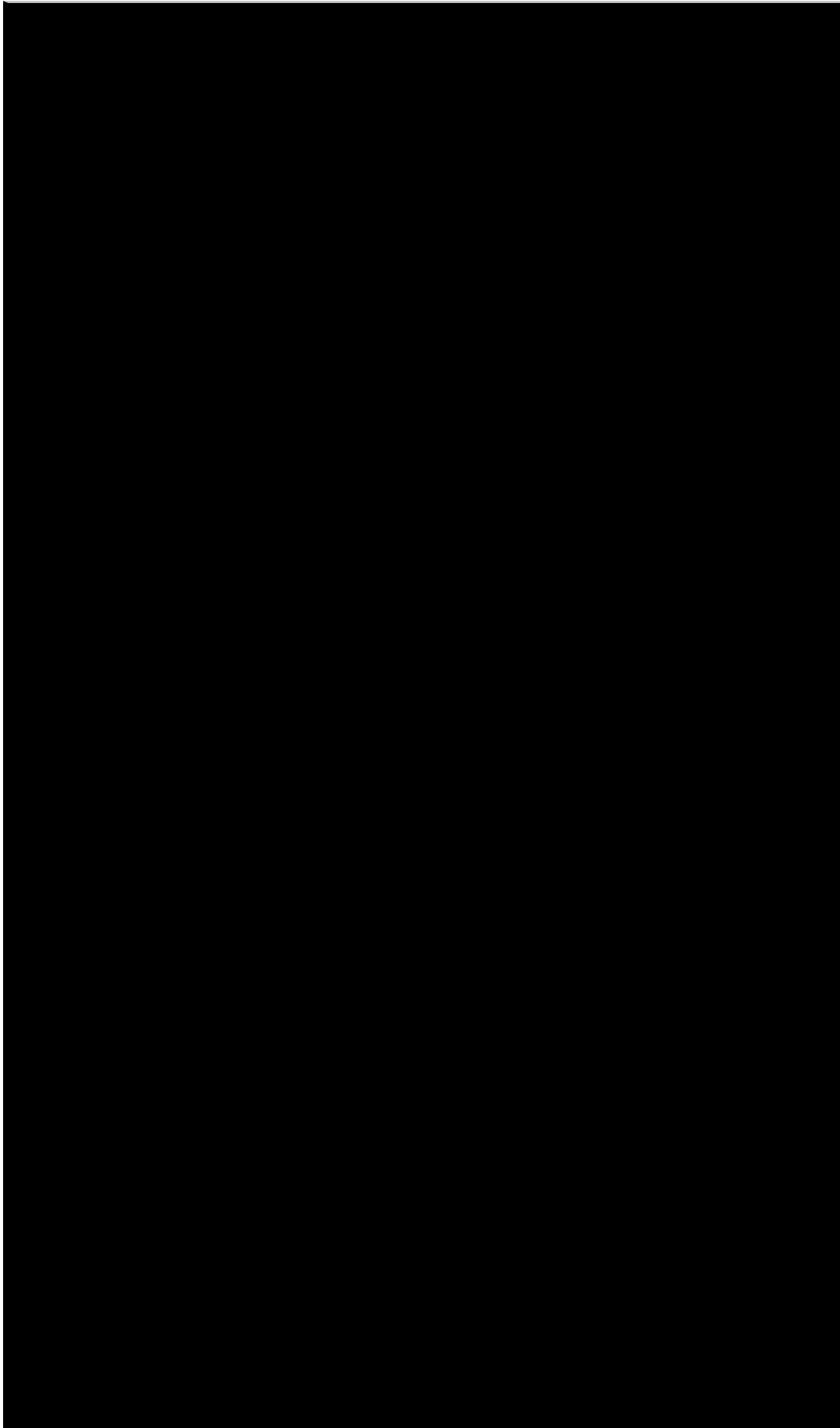


Figura 25.4: Parser, vista larga (material original Sinergia)

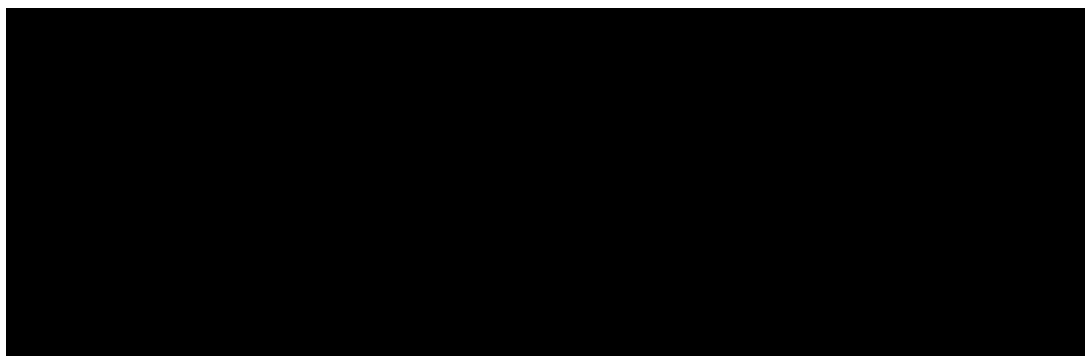


Figura 25.5: Parser recursivo en detalle (material original Sinergia)



Figura 25.6: Evaluación semántica, continuación (material original Sinergia)

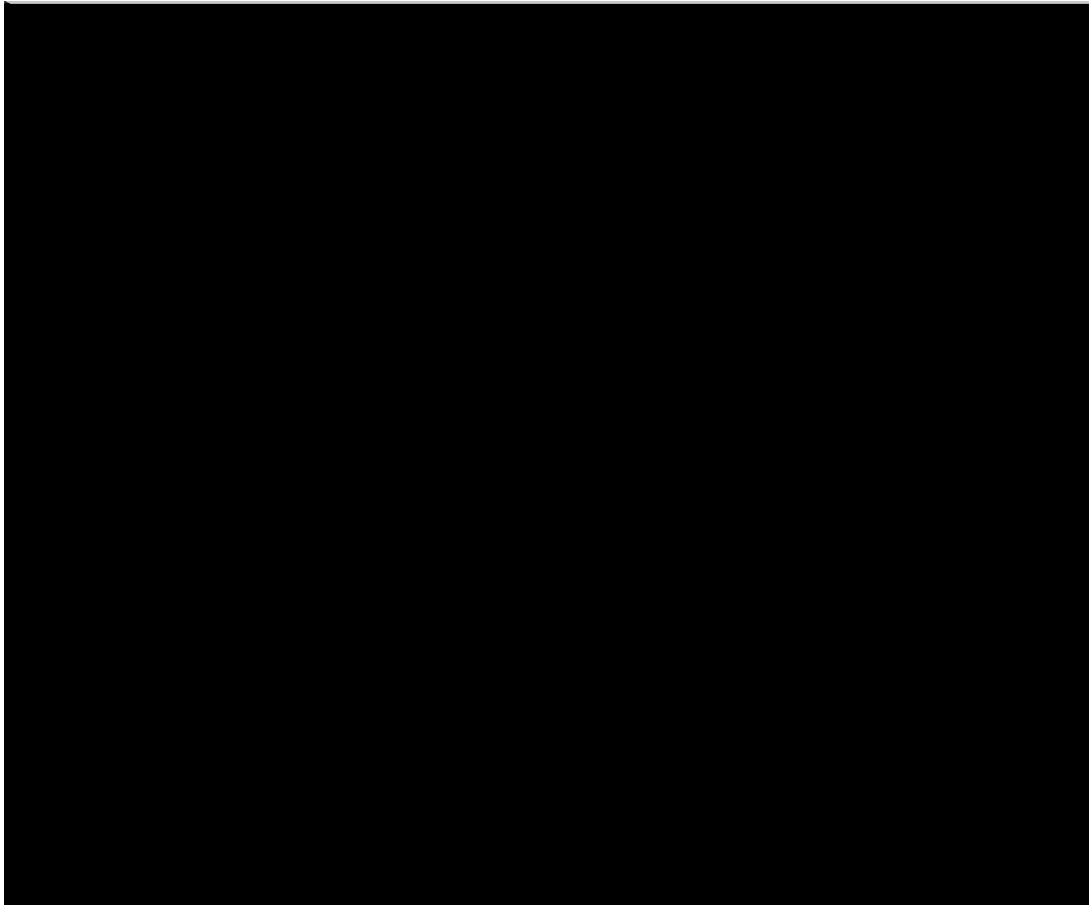


Figura 25.7: Evaluación y ramas (material original Sinergia)

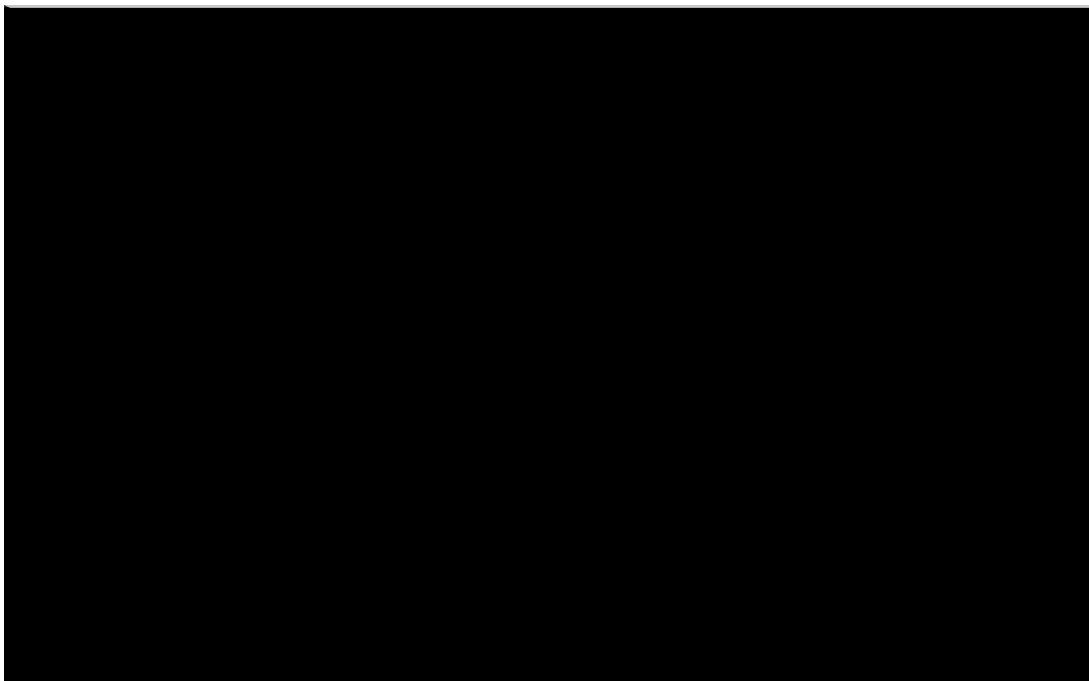


Figura 25.8: Envoltura del árbol a Unicode (material original Sinergia)



Figura 25.9: Estado reactivo de la vista (material original Sinergia)

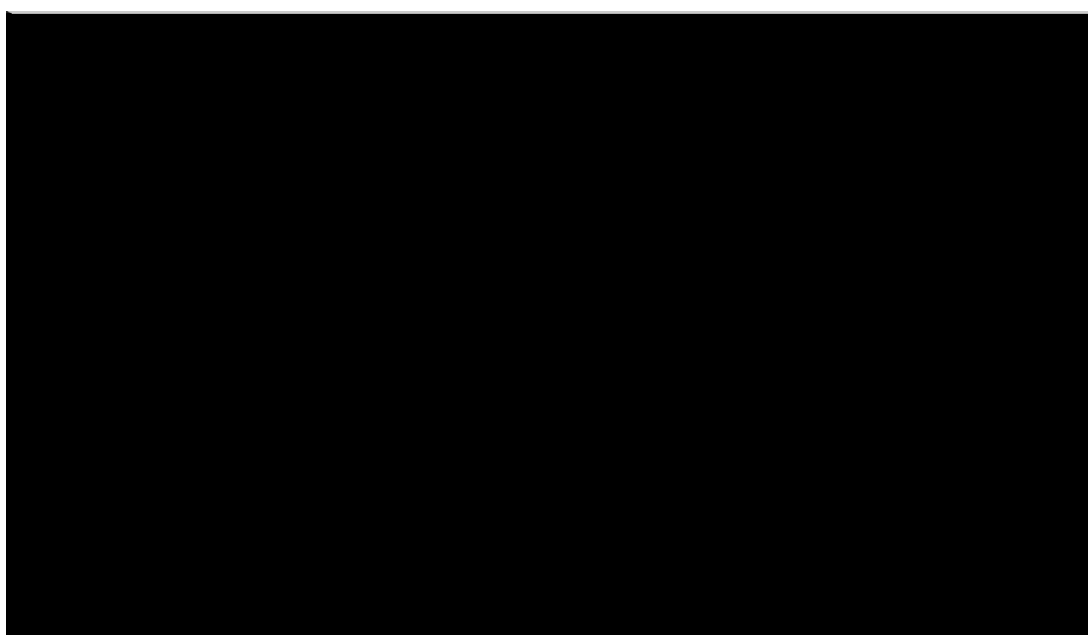


Figura 25.10: Estado reactivo, continuación (material original Sinergia)



Figura 25.11: Propiedades computadas (material original Sinergia)



Figura 25.12: Acceso documentado (material original Sinergia)



Figura 25.13: Pruebas del evaluador (material original Sinergia)

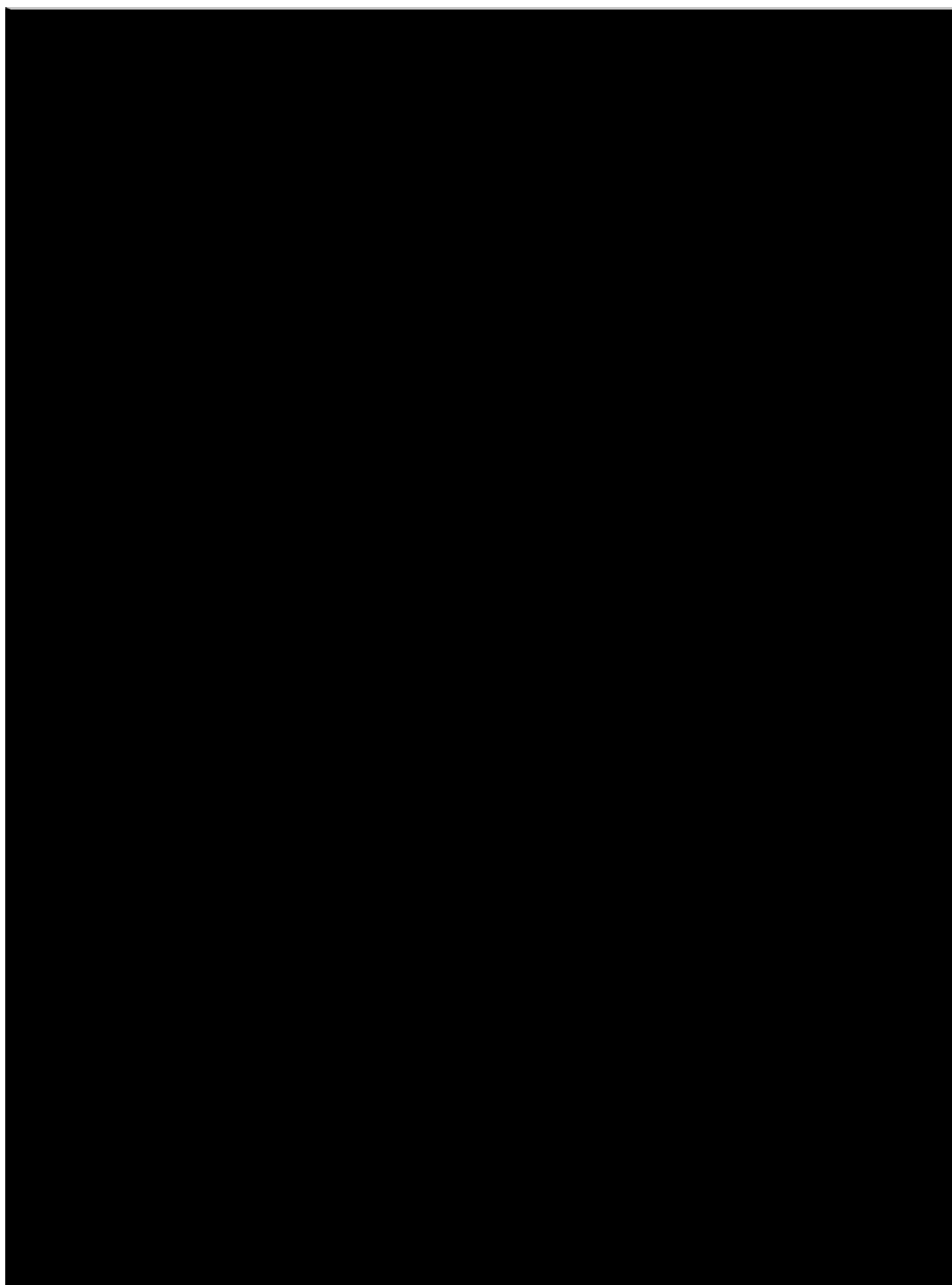


Figura 25.14: Árbol del repositorio (material original Sinergia)

Motores de conjuntos y cuantificadores

Define tus conjuntos

Universo U: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Conjunto A: 1, 2, 3, 4

Conjunto B: 3, 4, 5, 6

Selecciona una operación

$A \cup B$ $A \cap B$ $A - B$ $B - A$ A' B' $P(A)$

Diagrama de Venn

Resultado

Unión: $A \cup B$

{ 1, 2, 3, 4, 5, 6 }

Propiedades

$A \subseteq B$ ☒ No

$B \subseteq A$ ☒ No

$A = B$ ☒ No

Disjuntos ☒ No

Verificar pertenencia

Escribe un elemento...

Figura 25.15: Boceto inicial de conjuntos (material original Linus)

Nombre	Mensaje del último commit	Fecha del último...
...		
01_Propuesta	docs: agregar la propuesta del Sprint 1 y los registros de tareas motoras del Sprint 2	hace 3 días
02_Motor	tarea: fusionar upstream/main en sprint-3-hijos-de-linus	ayer
03_UI	tarea: fusionar upstream/main en sprint-3-hijos-de-linus	ayer
04_Despliegue	tarea: fusionar upstream/main en sprint-3-hijos-de-linus	ayer
ESTADO_EQUIPOS.md	docs: agrega PRs #9, #10, #11 al registro de cambios (Equipo Linus)	anteayer

Figura 25.16: Flujo Git del equipo Linus (material original)

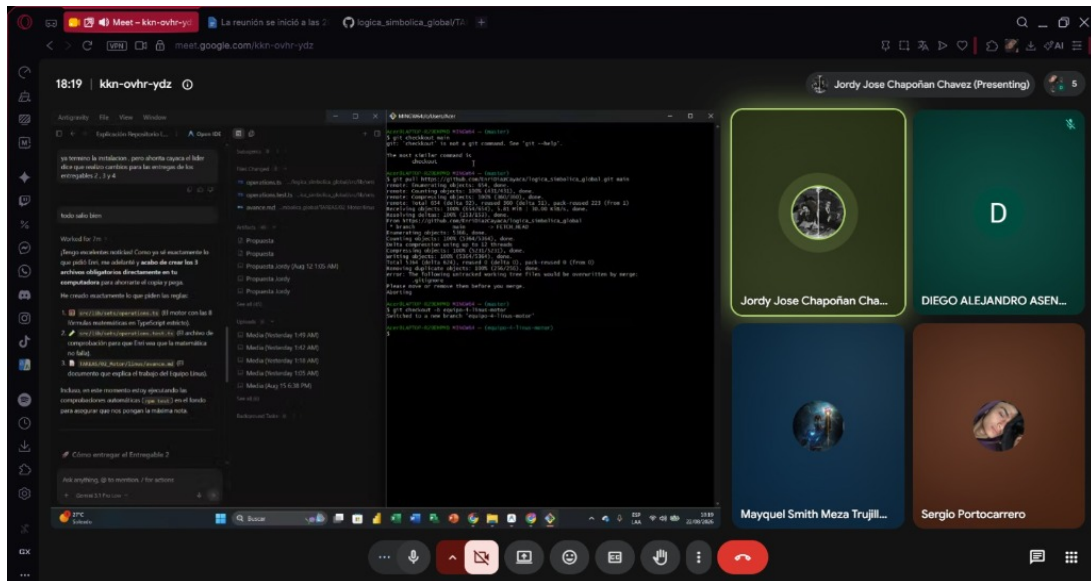


Figura 25.17: Trabajo colaborativo Linux (material original)

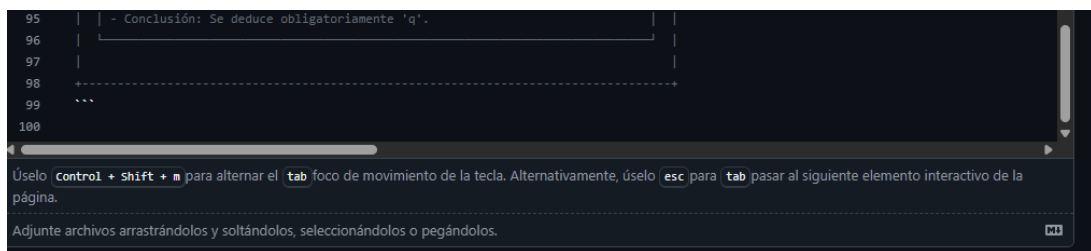


Figura 25.18: Boceto QuantifWeb (material original Modus Innova)

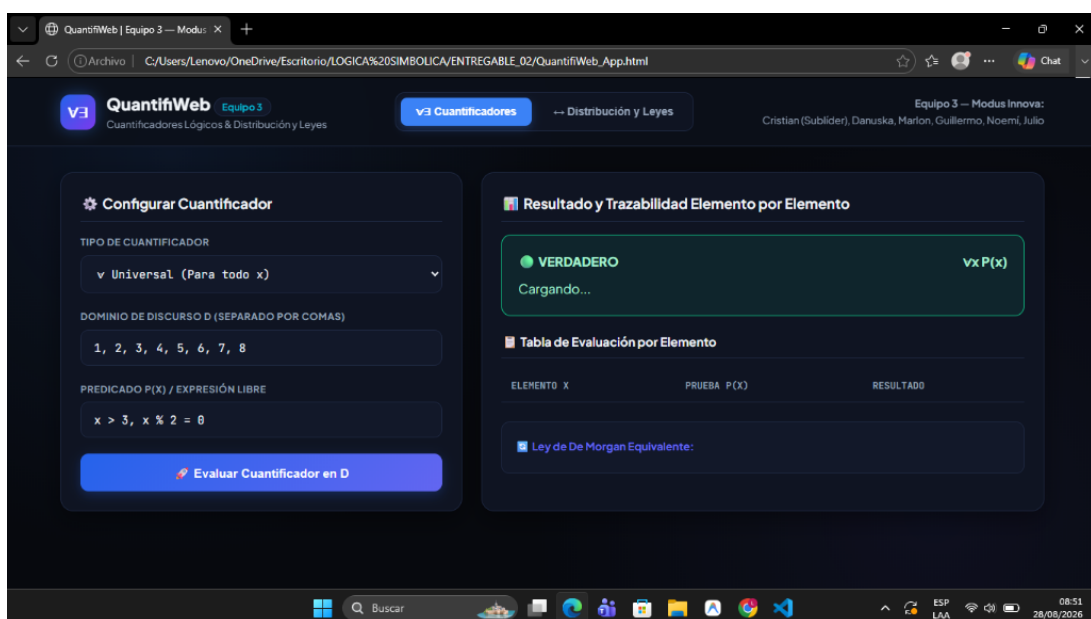


Figura 25.19: Motor de cuantificadores (material original Modus Innova)

Sucursales

Descripción general **Tuyo** Activo Duro Todo

Buscar sucursales...

Rama	Actualizado	Verificar estado	Detrás / Adelante	Solicitud de extracción
Parche Cristian-Lien-4	Hace 9 horas		0 1	
Parche Cristian-Lien-3	la semana pasada		75 1	
Parche 2 de Cristian-Lien	Hace 2 semanas		77 0	
Parche Cristian-Lien-1	Hace 2 semanas		105 1	11 # 3

Figura 25.20: Integración Vue (material original Modus Innova)

1) $(\exists x)[\sim Px \leftrightarrow (Qx \vee \sim Rx)]$

$\equiv (\exists x)[(\sim Px \rightarrow (Qx \vee \sim Rx)) \wedge ((Qx \vee \sim Rx) \rightarrow \sim Px)]$... Definición de Bicondicional

$\equiv (\exists x)[(Px \vee (Qx \vee \sim Rx)) \wedge ((Qx \vee \sim Rx) \rightarrow \sim Px)]$... Definición de Implicación y Doble Negación

$\equiv (\exists x)[(Px \vee (Qx \vee \sim Rx)) \wedge \sim((Qx \vee \sim Rx) \vee \sim Px)]$... Definición de Implicación y Doble Negación

Figura 25.21: Leyes, segunda parte (material original Modus Innova)

Validación, trazabilidad y páginas

El validador unifica más de 15 notaciones hacia claves canónicas con prevención de inyecciones. La trazabilidad acumula pasos en orden con instantáneas. Las nueve vistas orquestan entradas reactivas y motores sin contener lógica. El despliegue publica en Pages con vistas previas por Pull Request.

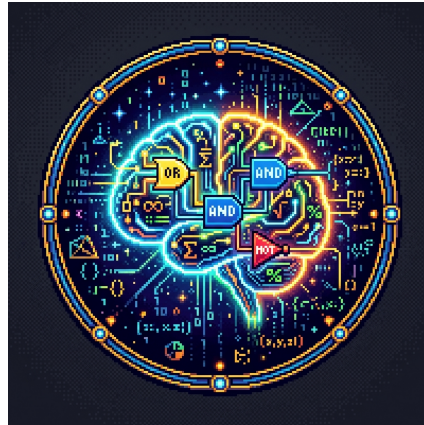


Figura 26.1: Logo institucional LogiLearn

Capítulo 27

Registro de decisiones de diseño

En síntesis: Vue 3 por componentes tipados; TypeScript estricto para motores; sin almacén global por estado local suficiente; doble parser (caracteres frente a palabras) por dominios distintos; cotas de 12 iteraciones y 2000 elementos contra la explosión; contenido externalizado para edición docente sin código.

Tabla 27.1: Ocho decisiones con alternativa descartada

Decisión	Elegido	Descartado
Framework	Vue 3 tipado	Alternativas sin tipado
Almacén	Estado local	Almacén global (innecesario)
Parser tablas	Por caracteres	Por palabras (ambiguo con $\wedge$)
Parser solver	Por palabras	Por caracteres (ilegible en español)
Dominio	Híbrido + estricto	Solo libre (inseguro)
Reescritura	Texto plano dirigido	Árbol completo (costoso)
Iteraciones	12 encadenadas	Ilimitadas (explosión)
Contenido	Claves por módulo	Literales en vistas

Capítulo 28

Despliegue y mantenimiento

Procedimiento paso a paso

1. Verificar tipos y empaquetar 1860 módulos con la base `/logica_simbolica_global/`.
2. Publicar la compilación en la rama de páginas con redirección para rutas.
3. Cada Pull Request genera una vista previa temporal con URL única.
4. Ante contenido antiguo, recarga forzada con `Ctrl + Shift + R`.

La compilación verifica tipos y empaqueta 1860 módulos. La base `/logica_simbolica_global/` sirve en Pages con redirección para rutas y vistas previas por Pull Request. El mantenimiento edita claves por módulo con interruptor de modo literal y valida con 189 pruebas antes de publicar.

Capítulo 29

Glosario técnico

AST Árbol de sintaxis abstracta por precedencia.

Precedencia Orden $\neg > \wedge > \vee > \rightarrow > \leftrightarrow$.

Forward chaining Encadenamiento hacia adelante hasta 12 iteraciones.

Contraejemplo Asignación con premisas V y conclusión F .

Testigo Elemento que satisface un $\exists$.

$D \subset \mathbb{Z}$ Dominio finito de enteros.

Δ Disyunción fuerte $(p \vee q) \wedge \neg(p \wedge q)$.

Modo literal Interruptor por módulo entre símbolos y palabras.

Trazabilidad Pasos acumulados en orden con instantáneas.

Potencia $P(A)$ con 2^n subconjuntos y máscara de bits.

Capítulo 30

Casos de prueba nominales por archivo

Cada fila reproduce el nombre declarado del caso en el repositorio. La suite total suma 189 pruebas en 17 archivos.

Tabla 30.1: Casos nominales (selección de)

Archivo	Caso
components/inferencias/___tests___/ArbolAST.test.ts	renderiza un árbol recursivo con <u>
components/inferencias/___tests___/ArbolAST.test.ts	respeto la precedencia de paréntesis
components/inferencias/___tests___/ArbolAST.test.ts	muestra un aviso cuando la sintaxis
components/inferencias/___tests___/ArbolAST.test.ts	normaliza notación matemática (P
components/inferencias/___tests___/ArbolAST.test.ts	muestra un árbol de ejemplo cuando
components/inferencias/___tests___/ArbolAST.test.ts	combina premisas y conclusión en U
components/inferencias/___tests___/FormularioInferencia.test.ts	deshabilita el botón de Demostrar s
components/inferencias/___tests___/FormularioInferencia.test.ts	habilita el botón si hay premisas y c
components/inferencias/___tests___/FormularioInferencia.test.ts	deshabilita el botón si isLoading es
components/inferencias/___tests___/FormularioInferencia.test.ts	emite submit normalizando símbolo
components/inferencias/___tests___/FormularioInferencia.test.ts	inserta tokens al presionar los boto
components/inferencias/___tests___/FormularioInferencia.test.ts	borra el último carácter al pulsar el
components/inferencias/___tests___/IndicadorResultado.test.ts	no muestra nada si resultado es
components/inferencias/___tests___/IndicadorResultado.test.ts	renderiza caso válido demostrada
components/inferencias/___tests___/IndicadorResultado.test.ts	renderiza caso válido con método in
components/inferencias/___tests___/IndicadorResultado.test.ts	renderiza caso inválido refutado
components/inferencias/___tests___/IndicadorResultado.test.ts	renderiza caso error con mensaje pe
components/inferencias/___tests___/PanelTrazabilidad.test.ts	renderiza un mensaje si no hay pas
components/inferencias/___tests___/PanelTrazabilidad.test.ts	renderiza correctamente el diagnóst
components/inferencias/___tests___/PanelTrazabilidad.test.ts	renderiza correctamente los pasos y
components/inferencias/___tests___/PanelTrazabilidad.test.ts	despliega y oculta la explicación dic
components/inferencias/___tests___/PanelTrazabilidad.test.ts	exporta el markdown académico cor
components/inferencias/___tests___/PanelTrazabilidad.test.ts	exporta un documento LaTeX comp
components/inferencias/___tests___/TraductorLenguajeNatural.test.ts	detecta variables de las premisas y
components/inferencias/___tests___/TraductorLenguajeNatural.test.ts	actualiza la narración cuando el usu
lib/cuantificadores/engine.test.ts	parsea una cadena de números sepa
lib/cuantificadores/engine.test.ts	elimina espacios extra
lib/cuantificadores/engine.test.ts	filtra entradas vacías
lib/cuantificadores/engine.test.ts	parsea rango con <(excluyente)

Tabla 30.1: Casos nominales (cont)

Archivo	Caso
lib/cuantificadores/engine.test.ts	parsea rango con $\leq$ (inclusivo)
lib/cuantificadores/engine.test.ts	parsea rango descendente
lib/integracion_motor.test.ts	debe procesar un flujo completo: E
lib/integracion_motor.test.ts	debe manejar entradas inválidas sin
lib/integracion_motor.test.ts	debe procesar notaciones simbólicas
lib/sets/operations.test.ts	Unión: $A \cup B$ combina todos los ele
lib/sets/operations.test.ts	Unión: propiedad conmutativa $A \cup$
lib/sets/operations.test.ts	Intersección: $A \cap B$ devuelve solo e
lib/sets/operations.test.ts	Intersección: conjuntos disjuntos re
lib/sets/operations.test.ts	Diferencia: $A - B$ devuelve lo que e
lib/sets/operations.test.ts	Complemento: A
lib/solver/astVisual.test.ts	cuenta nodos y profundidad correct
lib/solver/astVisual.test.ts	reconoce la aridad unaria de NO
lib/solver/astVisual.test.ts	respeta paréntesis: NO $(P \rightarrow Q)$ tie
lib/solver/astVisual.test.ts	genera estadísticas coherentes
lib/solver/parser.test.ts	debe tokenizar correctamente varia
lib/solver/parser.test.ts	debe aislar paréntesis sin importar
lib/solver/parser.test.ts	debe construir AST para expresión
lib/solver/parser.test.ts	debe respetar la precedencia (NO P
lib/solver/parser.test.ts	debe respetar los paréntesis explícit
lib/solver/parser.test.ts	debe lanzar error de sintaxis ante e
lib/solver/solver.test.ts	debe identificar variables iguales (ca
lib/solver/solver.test.ts	debe rechazar variables diferentes
lib/solver/solver.test.ts	debe encontrar contraejemplo para
lib/solver/solver.test.ts	debe encontrar contraejemplo para
lib/solver/solver.test.ts	debe aplicar Silogismo Disyuntivo E
lib/solver/solver.test.ts	debe aplicar Silogismo Disyuntivo E
lib/trazabilidad/trazabilidad.test.ts	devuelve un array vacío al inicio
lib/trazabilidad/trazabilidad.test.ts	registra un paso y lo retorna con da
lib/trazabilidad/trazabilidad.test.ts	acumula múltiples pasos en orden
lib/trazabilidad/trazabilidad.test.ts	genera estadoActual con todas las l
lib/trazabilidad/trazabilidad.test.ts	procesa un resultado válido con pas
lib/trazabilidad/trazabilidad.test.ts	procesa un resultado inválido sin pa
lib/truth-table/evaluator.test.ts	parsea una variable simple
lib/truth-table/evaluator.test.ts	parsea negación
lib/truth-table/evaluator.test.ts	parsea conjunción
lib/truth-table/evaluator.test.ts	parsea implicación con símbolo $\rightarrow$
lib/truth-table/evaluator.test.ts	parsea bicondicional con símbolo $\leftrightarrow$
lib/truth-table/evaluator.test.ts	lanza error con expresión vacía
lib/validator/validator.test.ts	debe rechazar cadenas vacías
lib/validator/validator.test.ts	debe rechazar cadenas con solo espa
lib/validator/validator.test.ts	debe normalizar espacios múltiples
lib/validator/validator.test.ts	debe convertir variables y operador
lib/validator/validator.test.ts	CB-05: debe normalizar flechas de c
lib/validator/validator.test.ts	CB-06: debe normalizar bicondicion
lib/validator/validator_stress.test.ts	debe sanitizar y parsear expresiones
lib/validator/validator_stress.test.ts	debe manejar expresiones con hasta
lib/validator/validator_stress.test.ts	debe manejar cadenas con muchos c

Tabla 30.1: Casos nominales (cont)

Archivo	Caso
lib/validator/validator__stress.test.ts	debe admitir combinaciones mixtas
lib/validator/validator__stress.test.ts	debe sanitizar negaciones precedien
lib/validator/validator__stress.test.ts	debe rechazar negaciones sin operar
pages/inferencias/___tests___/index.test.ts	realiza el flujo completo de inferenc
pages/inferencias/___tests___/index.test.ts	permite cambiar entre pestañas de
pages/inferencias/___tests___/index.test.ts	muestra el diagnóstico en el panel c
pages/inferencias/___tests___/index.test.ts	muestra el mensaje de error cuando
test_ejemplo.test.ts	debe ejecutar pruebas correctament

Capítulo 31

Etapas del reescritor con ejemplo

Tabla 31.1: Ocho etapas con transformación de ejemplo

Etapas	Ejemplo
Doble negación	$\neg\neg p \Rightarrow p$
Disyunción fuerte	$p \Delta q \Rightarrow (p \vee q) \wedge \neg(p \wedge q)$
Bicondicional	$p \leftrightarrow q \Rightarrow (p \rightarrow q) \wedge (q \rightarrow p)$
Implicación	$p \rightarrow q \Rightarrow \neg p \vee q$
De Morgan	$\neg(p \wedge q) \Rightarrow \neg p \vee \neg q$
Distribución	$p \wedge (q \vee r) \Rightarrow (p \wedge q) \vee (p \wedge r)$
Cuantificador vacío	$\forall x P \Rightarrow P$ si x no ocurre
Simplificación repetida	aplica la misma regla hasta fijar

Capítulo 32

Demostraciones formales por tabla

Conectivos primitivos

Tabla 32.1: Tablas de los cinco conectivos

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$
V	V	F	V	V	V
V	F	F	F	V	F
F	V	V	F	V	V
F	F	V	F	F	V

El bicondicional $p \leftrightarrow q$ es V cuando ambos coinciden. La precedencia $\neg > \wedge > \vee > \rightarrow > \leftrightarrow$ elimina paréntesis redundantes.

De Morgan proposicional y cuantificacional

$\neg(p \wedge q)$ coincide con $\neg p \vee \neg q$ en las 4 filas; al negar, $\wedge$ se convierte en $\vee$. La versión cuantificacional $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$ se verifica por testigo y contraejemplo sobre D finito.

Leyes de conjuntos por extensión

$(A \cup B)^c = A^c \cap B^c$ se verifica elemento por elemento: x está a la izquierda si y solo si no está ni en A ni en B . La potencia cumple $|P(A)| = 2^{|A|}$ por inducción sobre la máscara binaria.

Capítulo 33

Retrospectiva por equipo

Sinergia consolidó el parser con 15 notaciones y la clasificación ternaria. Hijos de Linus unió deducción formal con evaluación exhaustiva y diagnóstico de siete falacias. Modus Innova restringió el dominio a $D \subset \mathbb{Z}$ con cota de 2000 y reescritura dirigida. Linus separó el motor puro de la vista reactiva con Venn exacto por recorte. Las cuatro bitácoras del anexo evidencian el avance semanal.

Capítulo 34

Sistema de contenido por módulo

Tabla 34.1: Claves de contenido por módulo

Módulo	Claves principales
Tablas	<code>tablas.header</code> , <code>.input</code> , <code>.info</code> , <code>.tablaTitulo</code> , <code>.explicacion</code>
Cuantificadores	<code>cuantificadores.header</code> , <code>.dominio</code> , <code>.predicado</code> , <code>.resultado</code>
Inferencias	<code>inferencias.titulo</code> , <code>.pestanas</code> , <code>.trazabilidad</code> , <code>.historial</code>
Conjuntos	<code>conjuntos.header</code> , <code>.operaciones</code> , <code>.diagrama</code> , <code>.resultado</code>
Aprender	<code>aprender.conectores</code> , <code>.etiquetasGrupo</code> , <code>.pasos</code>
Leyes	<code>leyes[12]</code> , <code>leyesPage</code>
Global	<code>global.marca</code> , <code>.nav</code> , <code>.footer</code>

Cada módulo tiene su interruptor de modo literal que elige símbolos ($\forall$, $\rightarrow$, $\wedge$) o palabras (para todo, entonces, y) sin tocar código.

Capítulo 35

Categorías del validador en resumen

Vacío y blancos, espaciado, mayúsculas, flechas, bicondicionales, conjunción y disyunción en notaciones ASCII, negaciones, paréntesis pegados, desbalanceados y vacíos, caracteres prohibidos y emojis, inyecciones HTML, subíndices, operadores consecutivos y negaciones anidadas: 21 categorías con comportamiento esperado de sanitizar o lanzar error con mensaje.

Capítulo 36

Guía de lectura y trabajo futuro

Cómo leer este informe

Los Capítulos 2 a 4 fijan teoría y arquitectura; los Capítulos 5 a 8 desarrollan cada motor con esquemas y figuras; los Capítulos 9 a 11 integran, prueban y despliegan; el anexo transcribe las bitácoras. Cada figura cita su material original.

Trabajo futuro

Exportación de tablas a CSV e imagen; modo cuestionario con tablas incompletas; verificador de equivalencias entre dos fórmulas; formas normales conjuntiva y disyuntiva automáticas; calificación por docente; lógica de primer orden completa con múltiples variables; y tutor adaptativo sobre el almacén de progreso.

Capítulo 37

Índice de correspondencia de figuras

Cada figura cita su material original sin recomposición. La tabla resume la correspondencia.

Tabla 37.1: Figuras y fuentes

Etiqueta	Pie
fig:gantt	Cronograma del equipo Sinergia (material original)
tab:capas	Arquitectura en tres capas
fig:estructura	Estructura de archivos del proyecto (material original Sinergia)
fig:tec-gramatica	Gramática del motor proposicional (material original Sinergia)
fig:tec-token	Tokenizador y normalización (material original Sinergia)
fig:tec-parser	Parser descendente recursivo (material original Sinergia)
fig:tec-filas	Generación de filas por bits (material original Sinergia)
fig:tec-ast	Boceto del árbol sintáctico (archivo original Hijos de Linus)
fig:tec-dominio	Validación del dominio entero (material original Modus Innova)
fig:tec-delta	Regla de disyunción fuerte (material original Modus Innova)
fig:tec-venn	Diagrama de Venn interactivo (material original Linus)
fig:tec-diario1	Propuestas Fernando y Mike (Linus, archivos originales)
fig:tec-diario2	Parser del equipo Sinergia, detalle (material original)
fig:tec-diario3	Propuestas Nio y Sergio (Linus, archivos originales)
fig:tec-diario4	Entorno de práctica del equipo Hijos de Linus (archivo original)
fig:tec-leyes-demo	Demostración paso a paso del motor de leyes (Modus Innova, material original)
fig:tec-computed	Propiedades computadas de la vista (material original Sinergia)
fig:tec-estado	Estado reactivo de la vista de tablas (material original Sinergia)
fig:tec-cron	Cronograma de sprints (material original Sinergia)
fig:tec-linus-boc	Bocetos del equipo Linus (archivos originales)
fig:tec-hijos-ui	Interfaz del equipo Hijos de Linus (archivo original)
fig:tec-g-sin-port	Portada del informe original Sinergia
fig:tec-g-sin-tipos	Tipos del evaluador (material original Sinergia)
fig:tec-g-sin-afd	Tokenizador autómatas (material original Sinergia)
fig:tec-g-sin-parserlong	Parser, vista larga (material original Sinergia)
fig:tec-g-sin-parserdet	Parser recursivo en detalle (material original Sinergia)
fig:tec-g-sin-eval2	Evaluación semántica, continuación (material original Sinergia)
fig:tec-g-sin-eval3	Evaluación y ramas (material original Sinergia)
fig:tec-g-sin-wrap	Envoltura del árbol a Unicode (material original Sinergia)
fig:tec-g-sin-react	Estado reactivo de la vista (material original Sinergia)
fig:tec-g-sin-react2	Estado reactivo, continuación (material original Sinergia)
fig:tec-g-sin-comp	Propiedades computadas (material original Sinergia)
fig:tec-g-sin-acc	Acceso documentado (material original Sinergia)
fig:tec-g-sin-tests	Pruebas del evaluador (material original Sinergia)
fig:tec-g-sin-tree	Árbol del repositorio (material original Sinergia)
fig:tec-g-lin-wire	Boceto inicial de conjuntos (material original Linus)
fig:tec-g-lin-git	Flujo Git del equipo Linus (material original)

Tabla 37.1: Figuras y fuentes (continuación)

Etiqueta	Pie
fig:tec-g-lin-meet	Trabajo colaborativo Linus (material original)
fig:tec-g-mod-boc	Boceto QuantifiWeb (material original Modus Innova)
fig:tec-g-mod-mot	Motor de cuantificadores (material original Modus Innova)
fig:tec-g-mod-vue	Integración Vue (material original Modus Innova)
fig:tec-g-mod-ley2	Leyes, segunda parte (material original Modus Innova)
fig:tec-logo	Logo institucional LogiLearn

Capítulo 38

Matriz de trazabilidad requisito-evidencia

Cada requisito acordado para esta entrega se rastrea a su sección de evidencia.

Tabla 38.1: Requisito y evidencia

Requisito	Evidencia
Voz única	Redacción unificada en todos los capítulos
Manual solo uso	Capítulos de uso sin código interno ni Excel
Técnico sin manual	Arquitectura, motores, sprints y anexos
Todas las imágenes originales	Galería y figuras con material sin recompresión
Plantilla reelegida e índice	Portada, TOC, figuras, tablas y bibliografía global
Ortografía y terminología	$D \subset \mathbb{Z}$ y conectivos unificados
Código necesario con diagramas	Esquemas curados y tablas de complejidad
Scratch fuera del repo	Solo transcripción del anexo, PDFs fuera
TAREAS como evidencia	Anexo de bitácoras fieles formateadas

Bibliografía

- [1] Copi, I. M., Cohen, C. y McMahon, K. (2017). *Introducción a la lógica* (14.^a ed.). Pearson.
- [2] Hurley, P. J. (2015). *A Concise Introduction to Logic* (12.^a ed.). Cengage.
- [3] Suppes, P. (1971). *Introducción a la lógica simbólica*. Continental.
- [4] Mendelson, E. (2015). *Introduction to Mathematical Logic* (6.^a ed.). Chapman & Hall.
- [5] Halmos, P. R. (1960). *Naive Set Theory*. Springer.
- [6] Vue.js Team. (2024). *Vue 3 Documentation*. <https://vuejs.org>
- [7] Vite Team. (2024). *Vite Documentation*. <https://vitejs.dev>